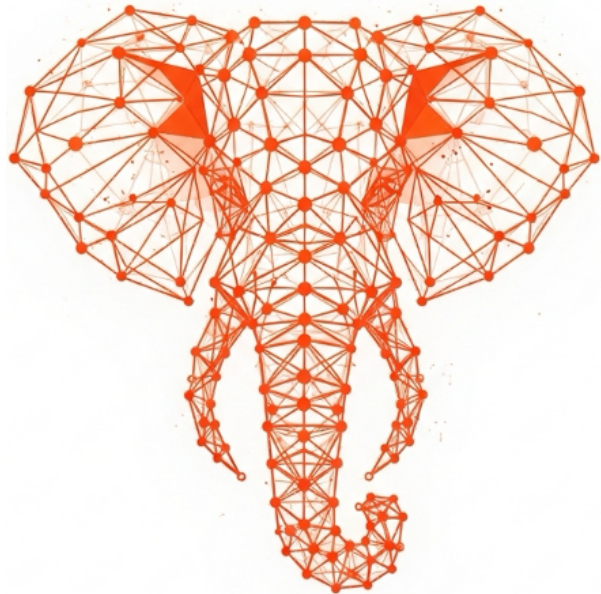




Uni
ct **DEEP LEARNING**
ADVANCED MODELS AND METHODS

Deep Learning

Advanced Models & Methods

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Laurea Magistrale in Informatica

Dip. di Matematica e Informatica

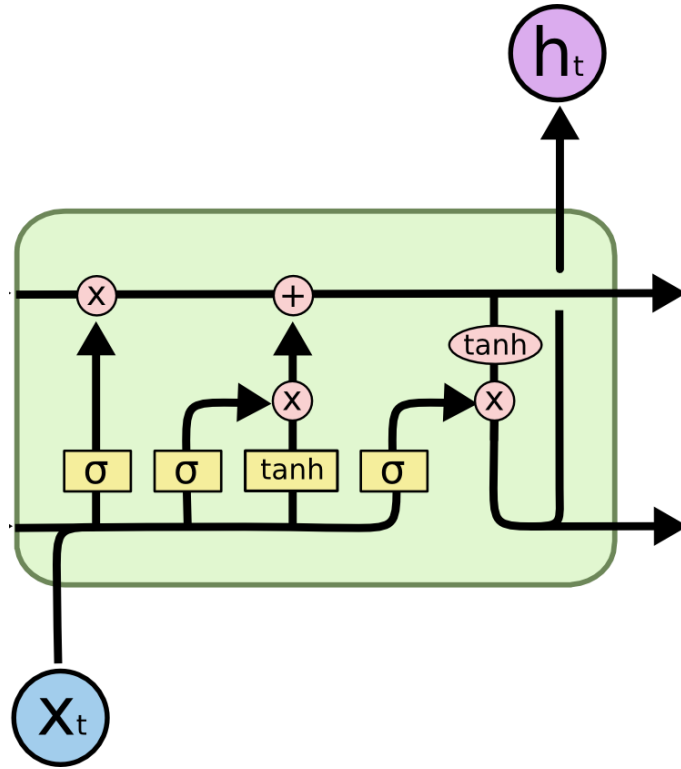
Università di Catania

Advanced Sequential Modeling

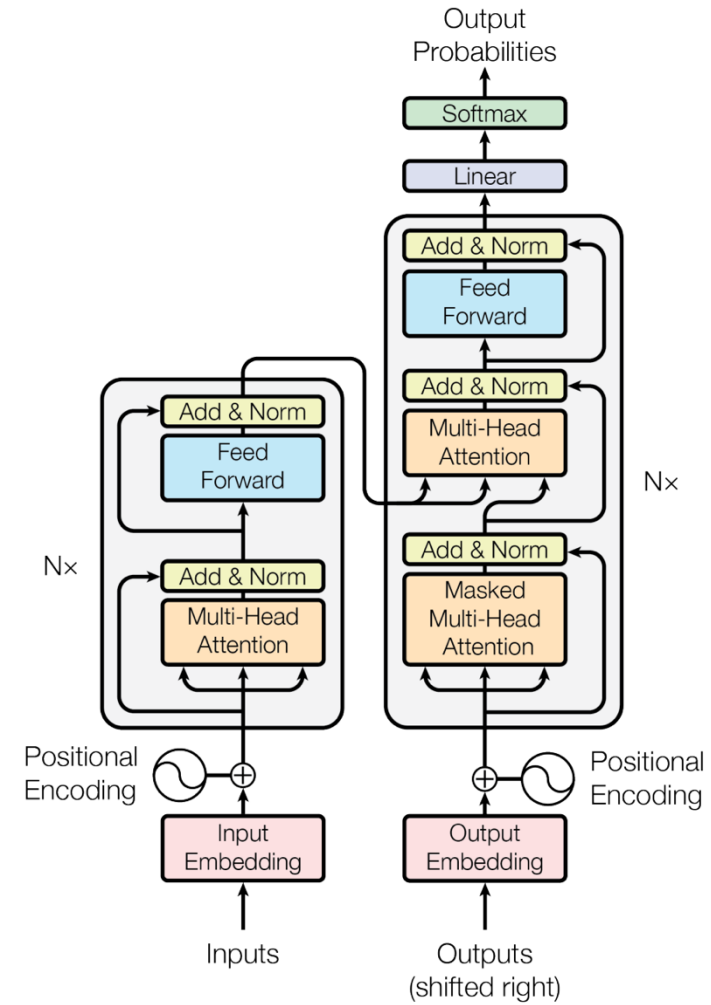
SEQUENCES

SEQUENCES EVERYWHERE

Recurrent Neural Networks vs Transformers



<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>



What About Convolutions?

An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

Shaojie Bai¹ J. Zico Kolter² Vladlen Koltun³

Abstract

For most deep learning practitioners, sequence modeling is synonymous with recurrent networks. Yet recent results indicate that convolutional architectures can outperform recurrent networks on tasks such as audio synthesis and machine translation. Given a new sequence modeling task or dataset, which architecture should one use? We conduct a systematic evaluation of generic convolutional and recurrent architectures for sequence modeling. The models are evaluated across a broad range of standard tasks that are commonly used to benchmark recurrent networks. Our results indicate that a simple convolutional architecture outperforms canonical recurrent networks such as LSTMs across a diverse range of tasks and datasets, while demonstrating longer effective memory. We conclude that the common association between sequence modeling and recurrent networks should be reconsidered, and convolutional networks should be regarded as a natural starting point for sequence modeling tasks. To assist related work, we have made code available at <http://github.com/locuslab/TCN>.

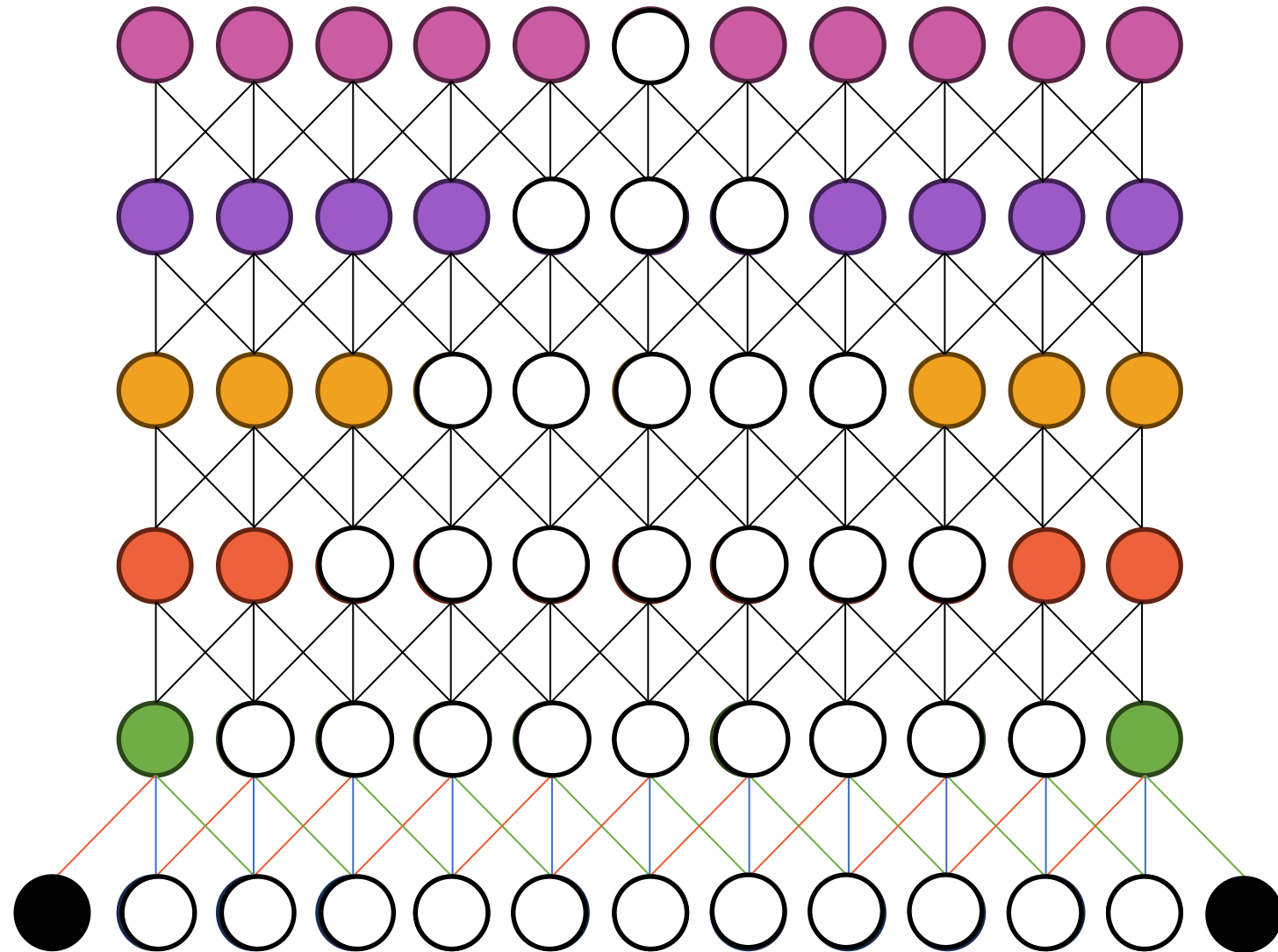
On the other hand, recent research indicates that certain convolutional architectures can reach state-of-the-art accuracy in audio synthesis, word-level language modeling, and machine translation (van den Oord et al., 2016; Kalchbrenner et al., 2016; Dauphin et al., 2017; Gehring et al., 2017a;b). This raises the question of whether these successes of convolutional sequence modeling are confined to specific application domains or whether a broader reconsideration of the association between sequence processing and recurrent networks is in order.

We address this question by conducting a systematic empirical evaluation of convolutional and recurrent architectures on a broad range of sequence modeling tasks. We specifically target a comprehensive set of tasks that have been repeatedly used to compare the effectiveness of different recurrent network architectures. These tasks include polyphonic music modeling, word- and character-level language modeling, as well as synthetic stress tests that had been deliberately designed and frequently used to benchmark RNNs. Our evaluation is thus set up to compare convolutional and recurrent approaches to sequence modeling on the recurrent networks’ “home turf”.

To represent convolutional networks, we describe a generic temporal convolutional network (TCN) architecture that is

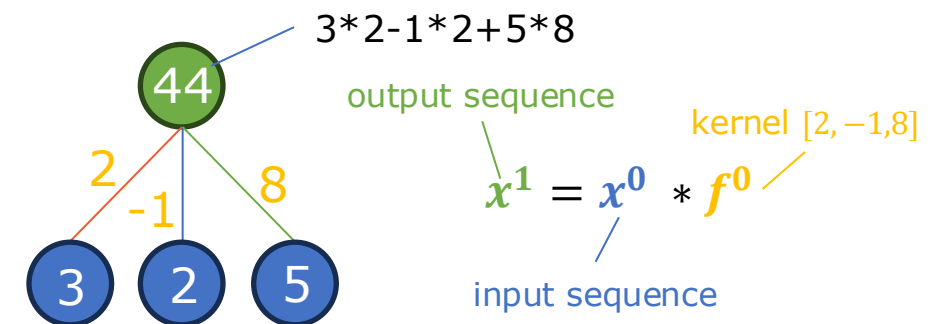
Shows empirically that temporal convolution networks works better than LSTMs in many sequence processing problems.

Temporal Convolutional Networks (TCN)



- Sequence-to-sequence model
- Learnable parameters are kernel weights
- Of course, nonlinear activation functions are placed in between layers 😊
- Highly parallelizable
- Growing receptive field

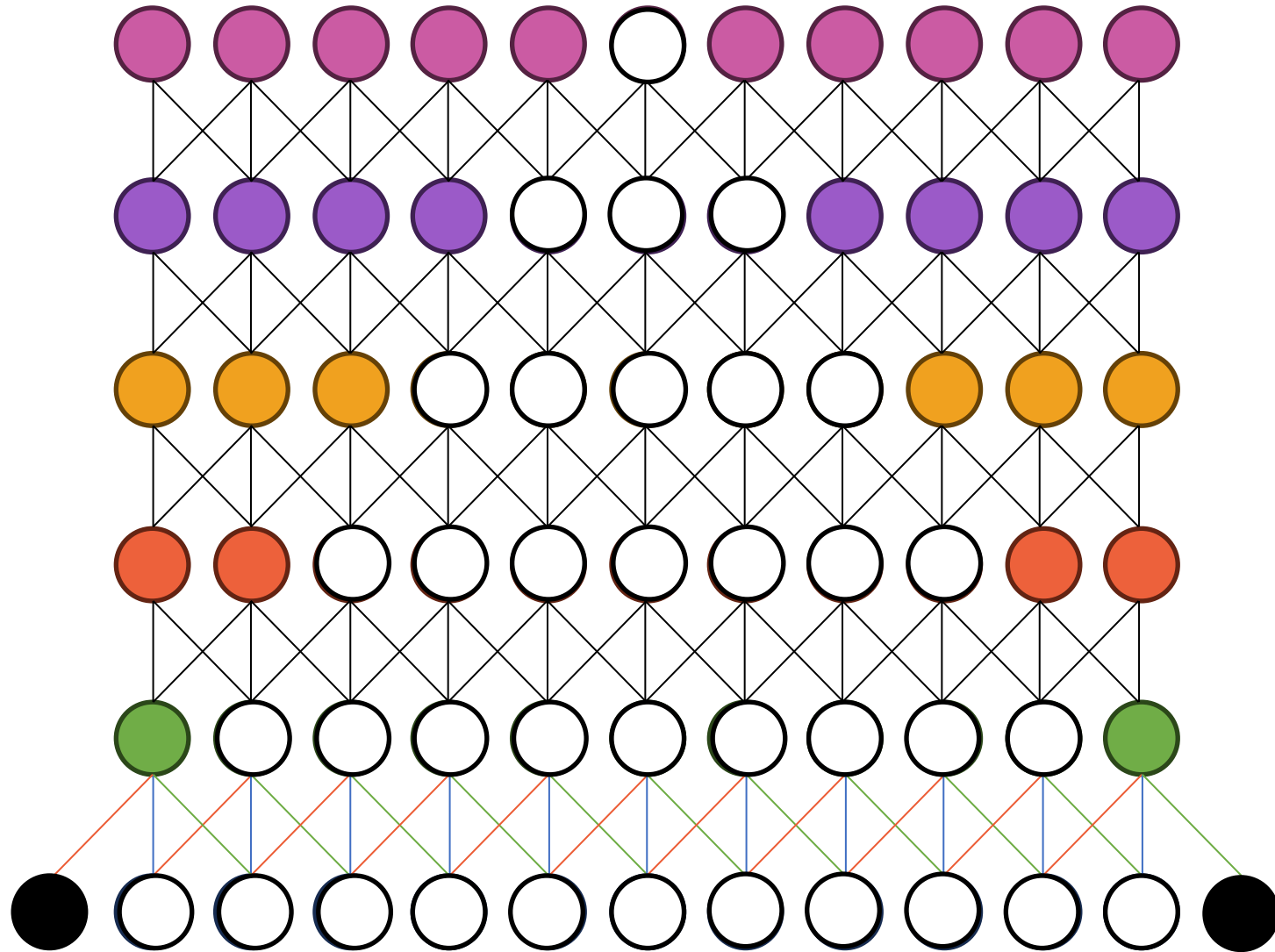
New layer: different kernel $x^2 = x^1 * f^1$



padding (0)

1D input sequence of length L - each element is a D -dimensional vector (e.g., feature frames): $BS \times L \times D$

Depth and sequence length



Problem: how many layers do we need to make sure that each unit can reason globally about the sequence?

It turns out that's:

$$d \geq \left\lceil \frac{L - k}{k - 1} + 1 \right\rceil$$

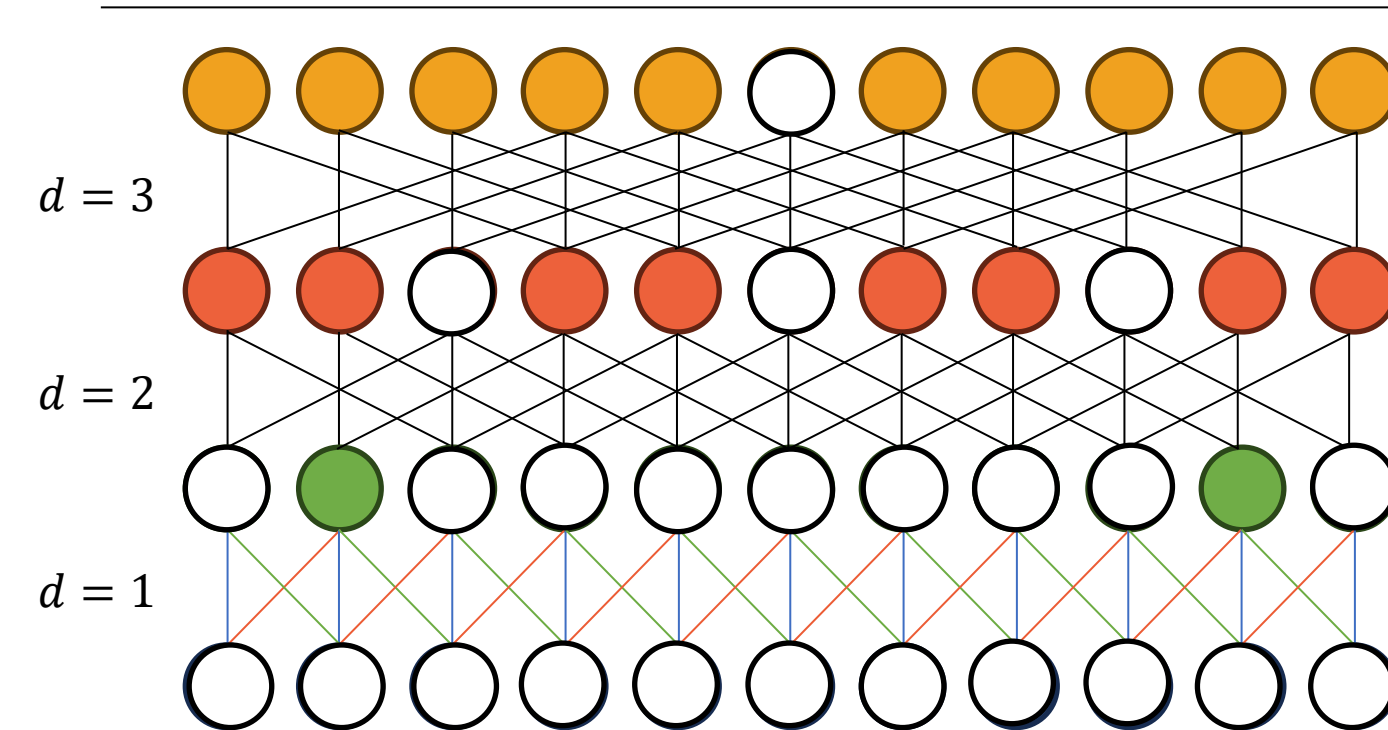
Where d is the number of layers (depth), k is the kernel size, and L is the sequence length.

For example:

$$L = 100, k = 3 \Rightarrow d \geq 50$$

Dilated Convolutions

- Three layers are enough to achieve a large receptive field.
- Without dilated convolutions, we needed 5 layers
- Dilated convolutions act like pooling in 2D CNNs



Idea: dilated convolutions

- Perform convolutions with a dilation factor d which skips $d - 1$ units to increase receptive field
- d increases with layers
- As a consequence, we need fewer layers to make sure that each unit «sees» the whole input

In practice:

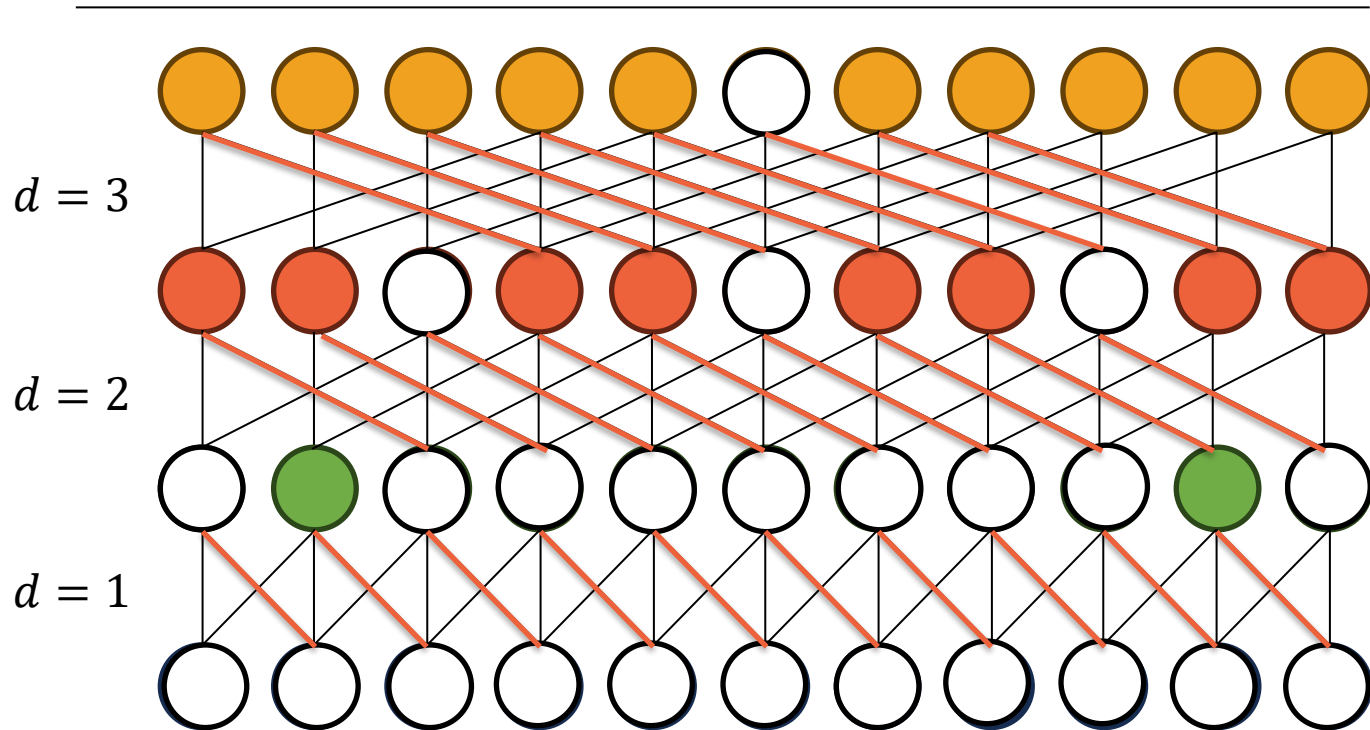
$$D \geq \left\lceil \log_2 \left(\frac{L-1}{k-1} + 1 \right) \right\rceil$$

For instance:

$$L = 100, k = 3 \Rightarrow d \geq 6$$

TCN and online processing

The white unit in the top layer sees the whole video, so the model can only be used for offline tasks, and cannot be deployed in an online or streaming fashion.



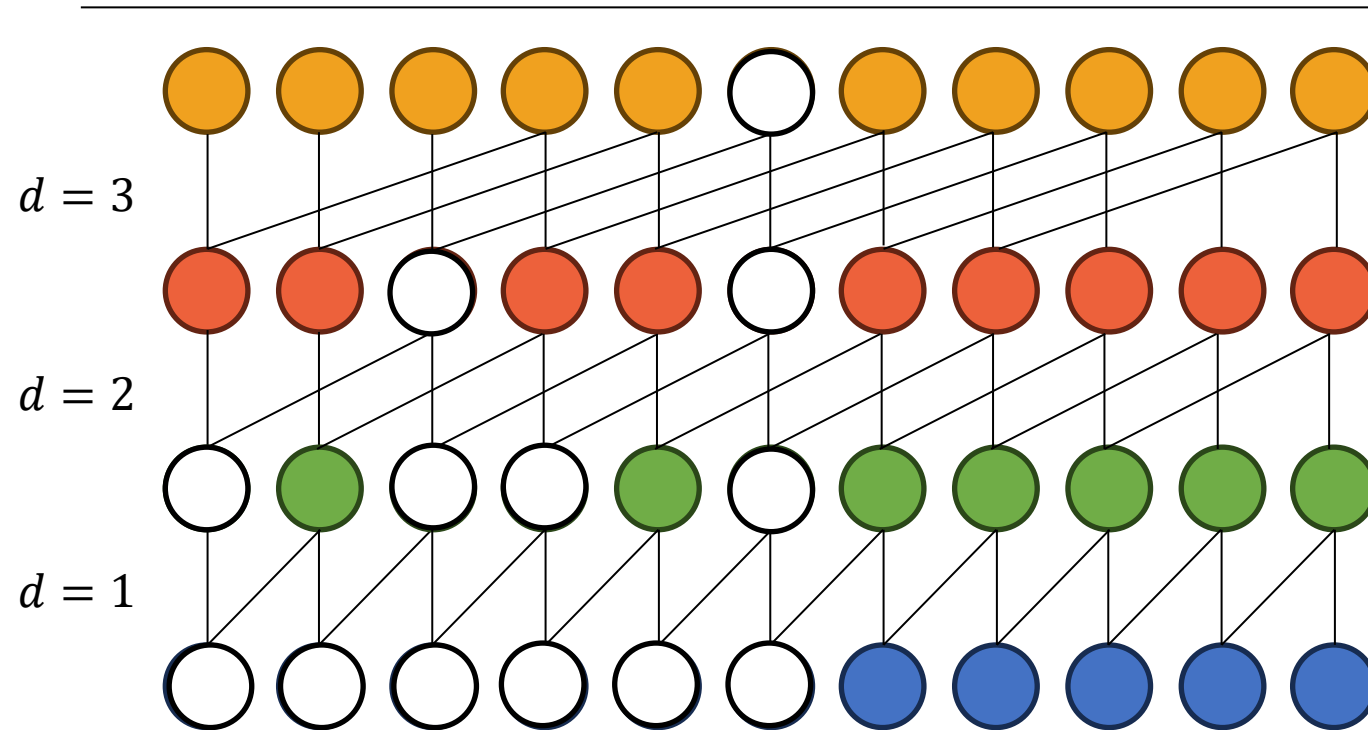
- As introduced, TCNs are good for «offline» tasks, where the video is processed in batch mode.
- Indeed, in the last layer, each unit actually «sees into the future», so it is necessary to have the whole video available in order to process it online.

Solution?

Cut the connections **looking into the future!**

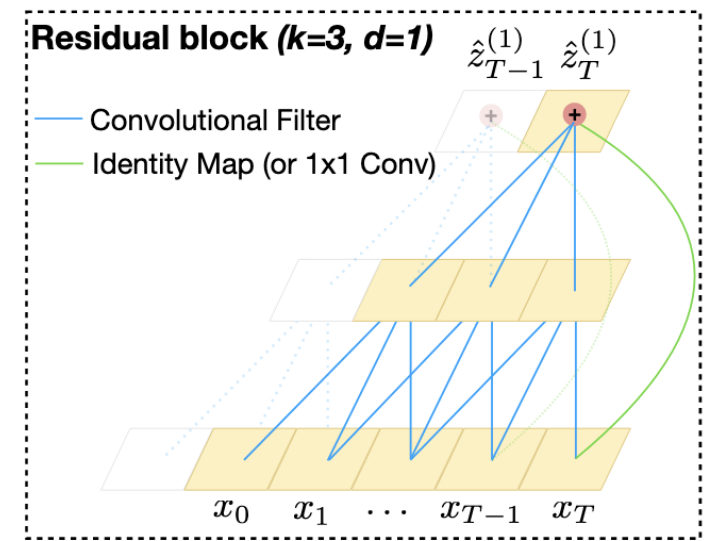
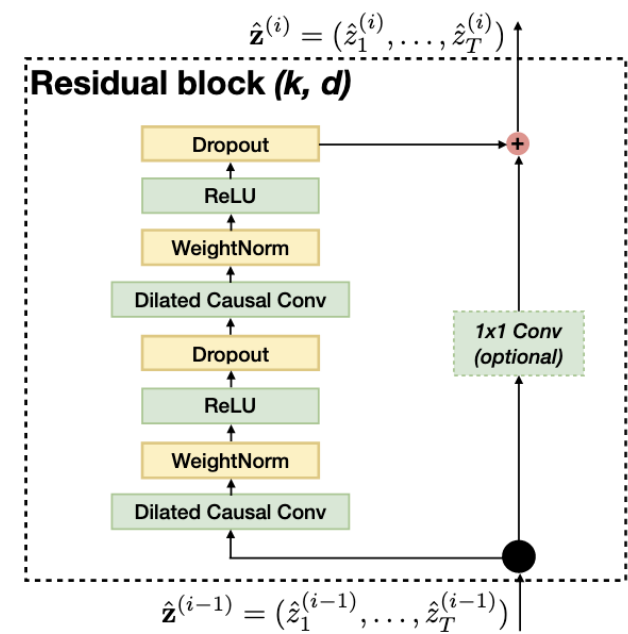
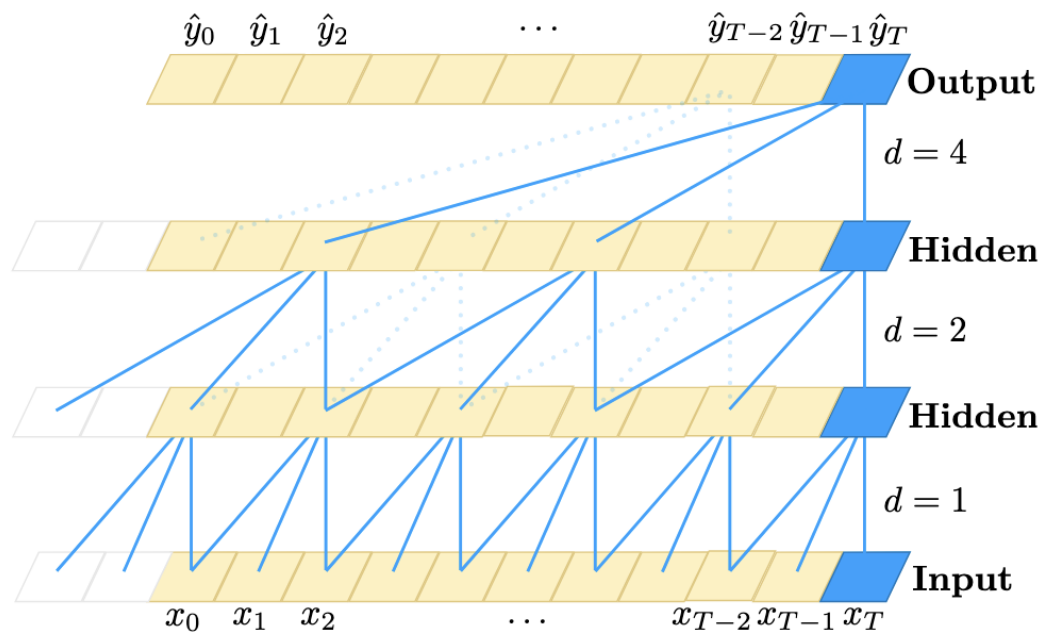
Dilated causal convolutions

With dilated causal convolutions, we keep the large receptive field, but each unit only looks at past units, hence making TCN suitable for online processing



- By removing forward looking connections, at each layer, each unit only "sees" past units.
- In this way, the whole processing is compatible with online/streaming scenarios.
- It should be noted that, while possible, streaming inference is still not super-straightforward and we need to keep a cache of past observations and activations.
- In practice, we obtain a causal TCN by using a smaller kernel (e.g., $k=2$) and shifting it backward.

Architectural Details



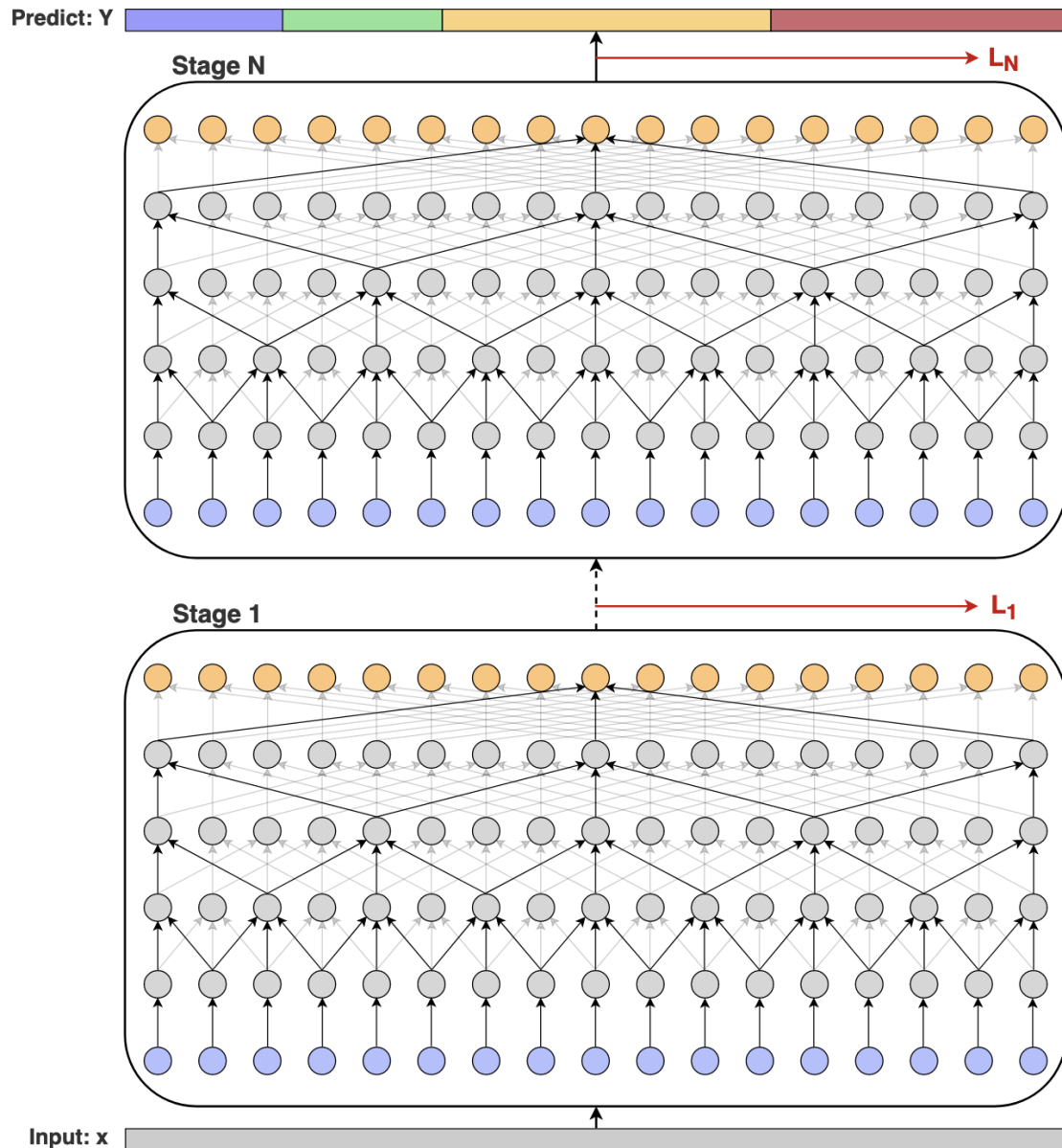
In practice, TCNs can be implemented in slightly different ways (as CNNs), including choices of:

- Activation functions
- Normalization techniques
- Residual connections

Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	TCN
Seq. MNIST (accuracy ^h)	70K	87.2	96.2	21.5	99.0
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	97.2
Adding problem $T=600$ (loss ^ℓ)	70K	0.164	5.3e-5	0.177	5.8e-5
Copy memory $T=1000$ (loss)	16K	0.0204	0.0197	0.0202	3.5e-5
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	8.10
Music Nottingham (loss)	1M	3.29	3.46	4.05	3.07
Word-level PTB (perplexity ^ℓ)	13M	78.93	92.48	114.50	88.68
Word-level Wiki-103 (perplexity)	-	48.4	-	-	45.19
Word-level LAMBADA (perplexity)	-	4186	-	14725	1279
Char-level PTB (bpc ^ℓ)	3M	1.36	1.37	1.48	1.31
Char-level text8 (bpc)	5M	1.50	1.53	1.69	1.45

In 2018, they were shown to be competitive with recurrent networks on many generic sequence processing tasks

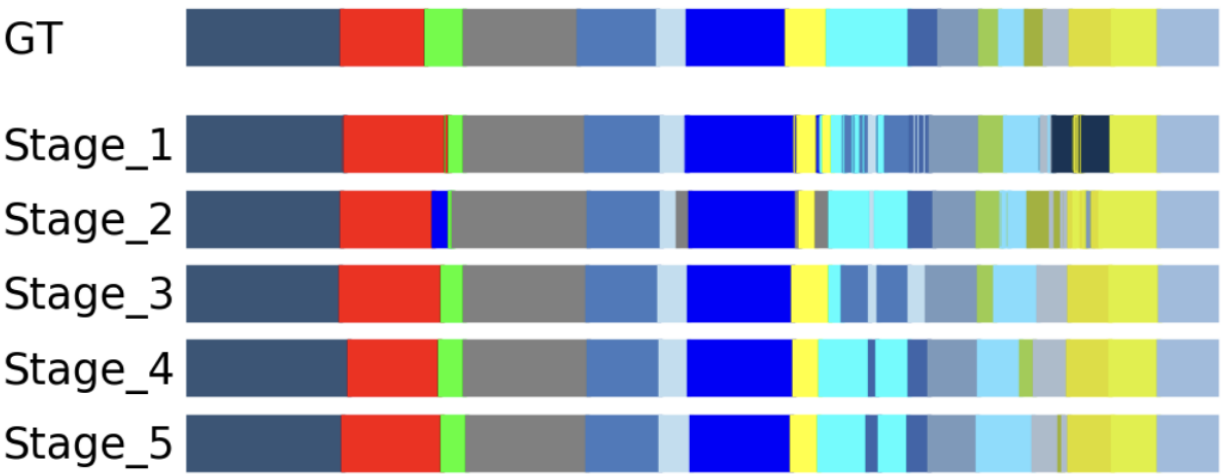
Multi-Stage TCN (MS-TCN) for action segmentation



- Pre-extracted features are used as input
- The model operates in «stages»
- In the first stage, the input is the sequence of features and the output is the output segmentation (actually, per-frame probability distributions)
- Subsequent stages take as input segmentations from previous stages and «refine» them, up till the last stage
- Intermediate losses are used ($L_1, \dots, L_N$) to make sure that intermediate outputs are indeed segmentations
- Uses a-causal convolutions (good for offline processing)

Multi-Stage TCN (MS-TCN) for action segmentaiton

Example of obtained progressive refinement:



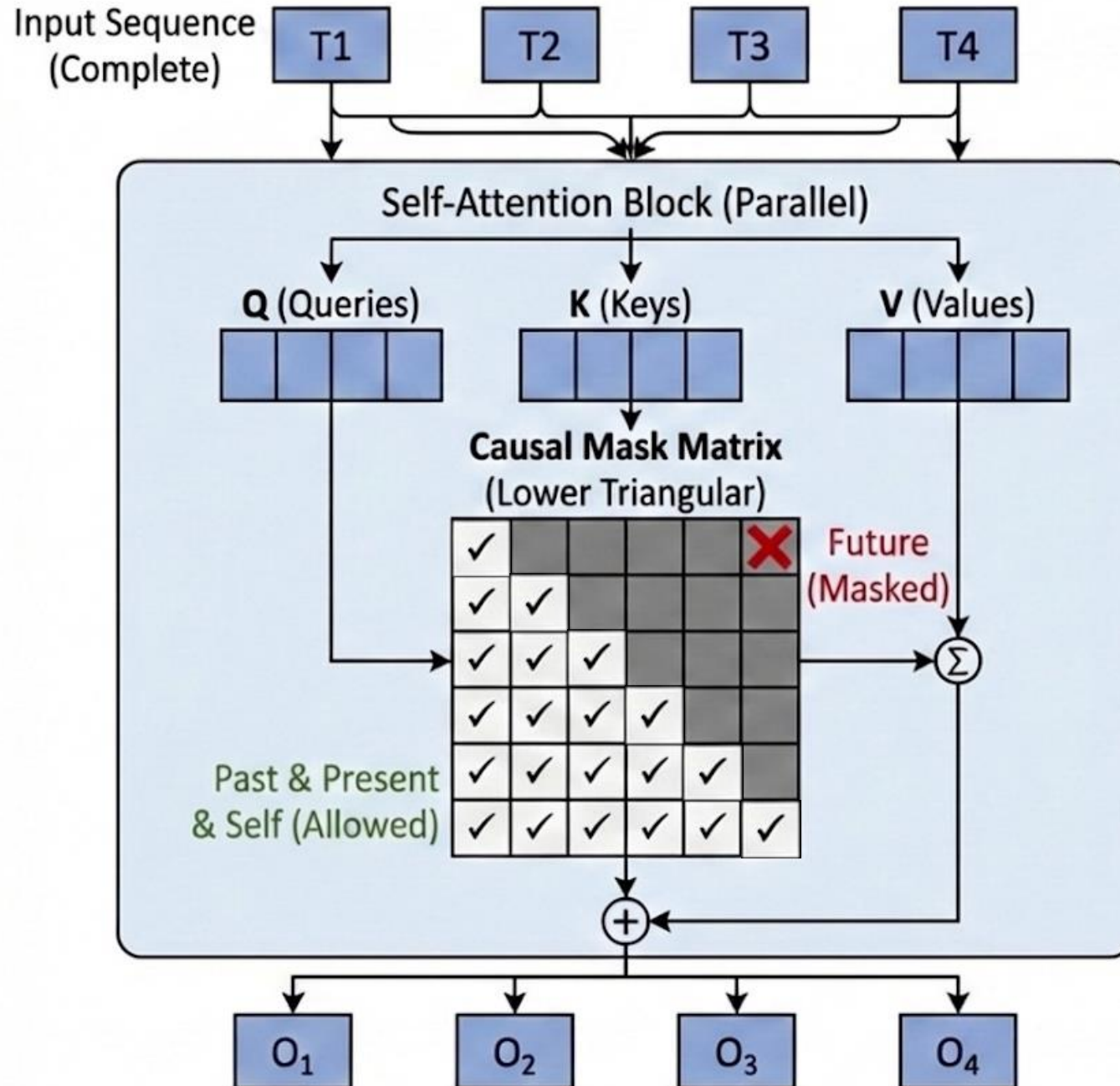
Effect of number of stages

	F1@{10,25,50}			Edit	Acc
SS-TCN	27.0	25.3	21.5	20.5	78.2
MS-TCN (2 stages)	55.5	52.9	47.3	47.9	79.8
MS-TCN (3 stages)	71.5	68.6	61.1	64.0	78.6
MS-TCN (4 stages)	76.3	74.0	64.5	67.9	80.7
MS-TCN (5 stages)	76.4	73.4	63.6	69.2	79.5

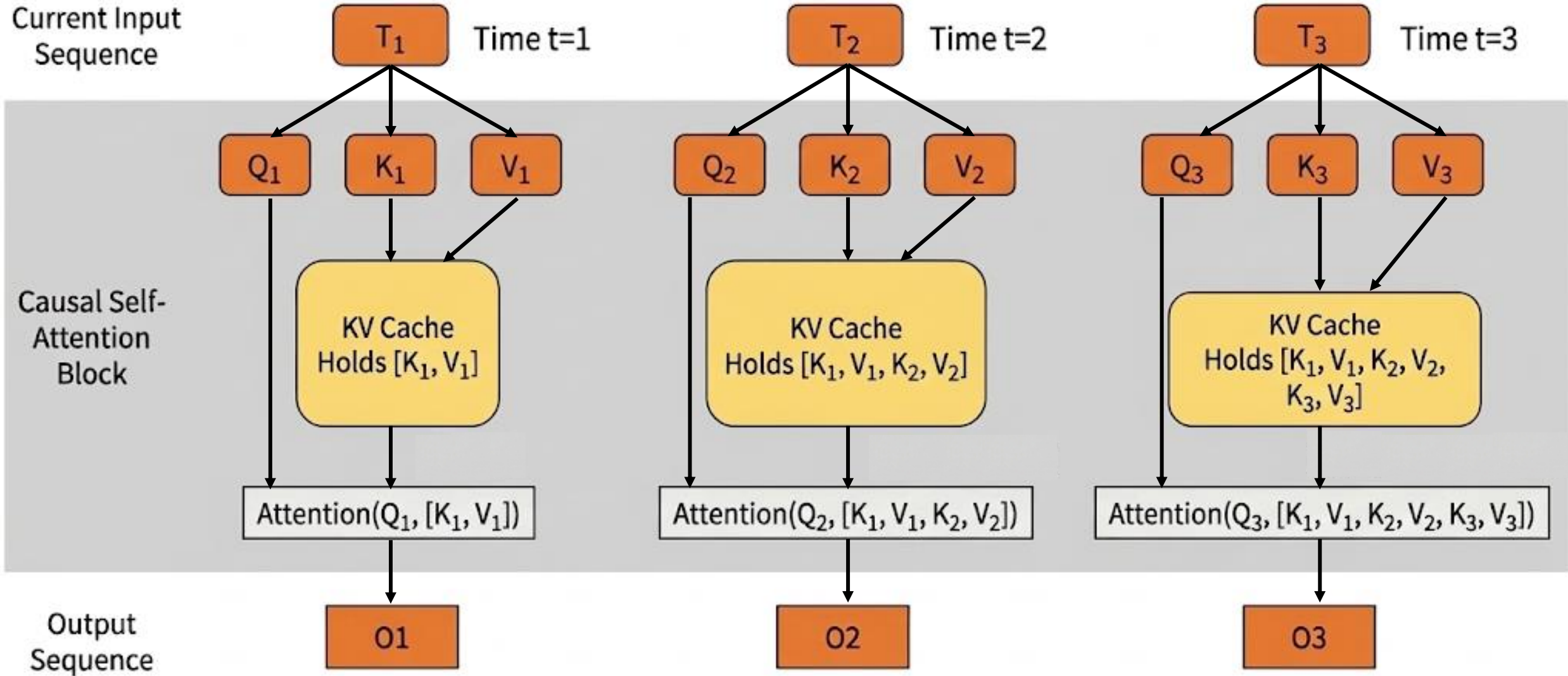
50Salads	F1@{10,25,50}			Edit	Acc
IDT+LM [20]	44.4	38.9	27.8	45.8	48.7
Bi-LSTM [23]	62.6	58.3	47.0	55.6	55.7
ED-TCN [15]	68.0	63.9	52.6	59.8	64.7
TDRN [17]	72.9	68.5	57.2	66.0	68.1
MS-TCN	76.3	74.0	64.5	67.9	80.7
GTEA	F1@{10,25,50}			Edit	Acc
Bi-LSTM [23]	66.5	59.0	43.6	-	55.5
ED-TCN [15]	72.2	69.3	56.0	-	64.0
TDRN [17]	79.2	74.4	62.7	74.1	70.1
MS-TCN	85.8	83.4	69.8	79.0	76.3
MS-TCN (FT)	87.5	85.4	74.6	81.4	79.2
Breakfast	F1@{10,25,50}			Edit	Acc
ED-TCN [15]*	-	-	-	-	43.3
HTK [14]	-	-	-	-	50.7
TCFPN [5]	-	-	-	-	52.0
HTK(64) [13]	-	-	-	-	56.3
GRU [21]*	-	-	-	-	60.6
MS-TCN (IDT)	58.2	52.9	40.8	61.4	65.1
MS-TCN (I3D)	52.6	48.1	37.9	61.7	66.3

Wait, isn't attention all we need?

Can Transformers be Used Online? yes, with causal masking

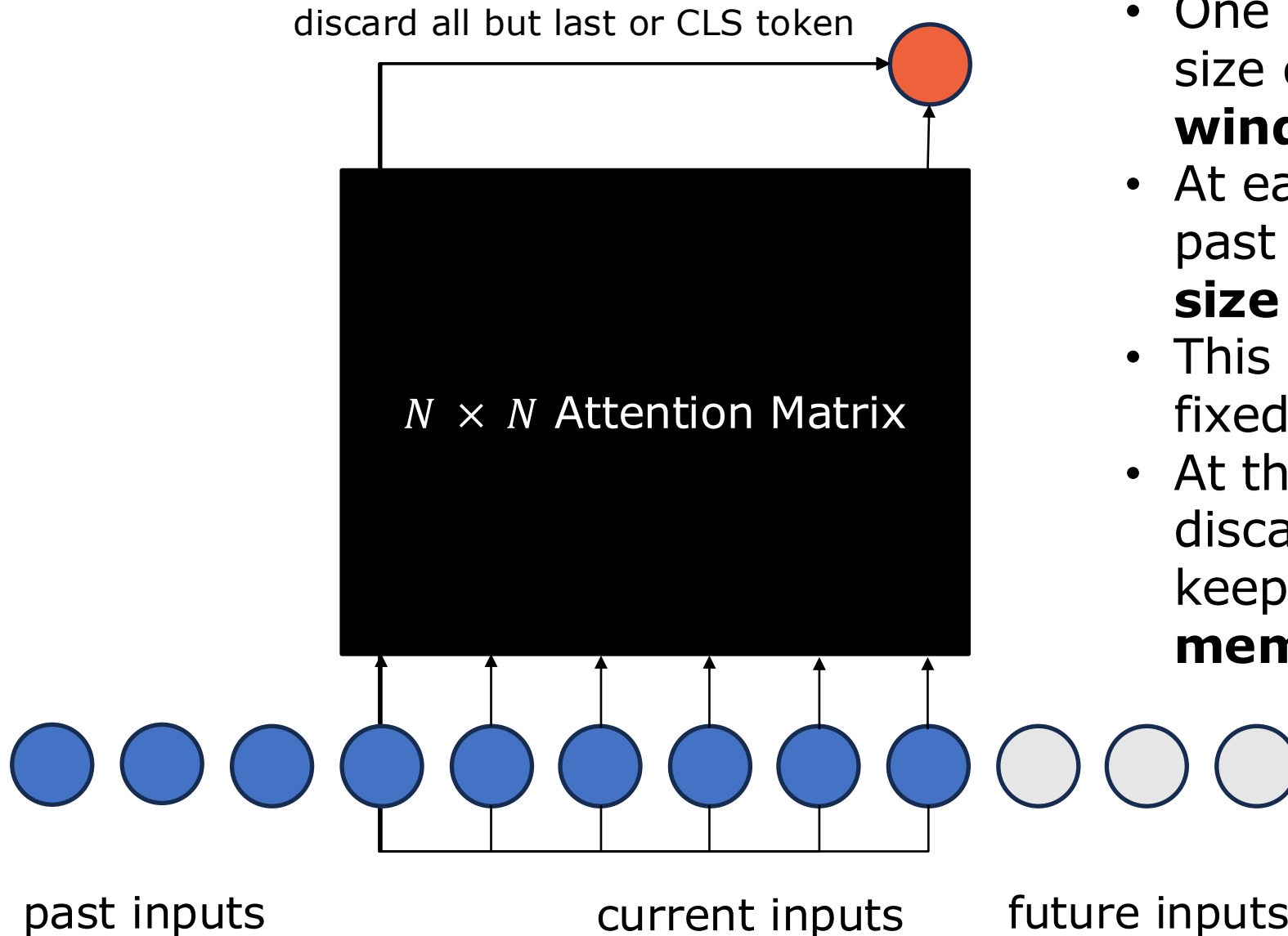


Online Processing in Practice (KV-Cache)



Problem: KV cache size and computation increase with time.

Sliding Window Transformer



- One way to keep the KV cache's size constant is to use a **sliding window**;
- At each time-step, we discard past inputs, keeping a **fixed size equal to N** ;
- This keeps the attention matrix fixed to **$N \times N$** ;
- At the same time, however, it discards past inputs, practically keeping only a «**short memory**» of past observations.

Long Short-Term Transformer for Online Action Detection

Mingze Xu Yuanjun Xiong Hao Chen Xinyu Li
Wei Xia Zhuowen Tu Stefano Soatto

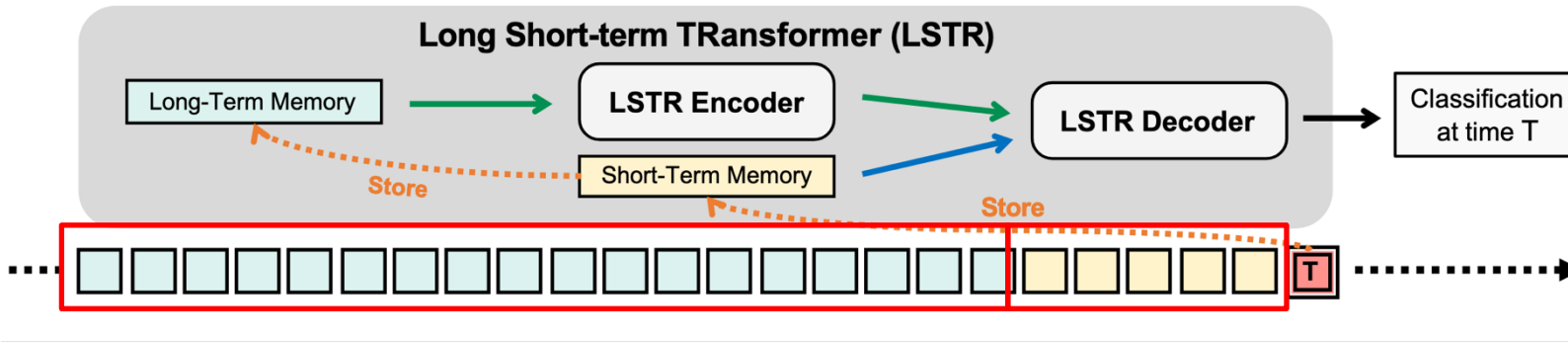
Amazon/AWS AI

{xumingze,yuanjx,hxen,xxnl,wxia,ztu,soattos}@amazon.com

Abstract

We present Long Short-term TRansformer (LSTR), a temporal modeling algorithm for online action detection, which employs a long- and short-term memory mechanism to model prolonged sequence data. It consists of an LSTR encoder that dynamically leverages coarse-scale historical information from an extended temporal window (*e.g.*, 2048 frames spanning of up to 8 minutes), together with an LSTR decoder that focuses on a short time window (*e.g.*, 32 frames spanning 8 seconds) to model the fine-scale characteristics of the data. Compared to prior work, LSTR provides an effective and efficient method to model long videos with fewer heuristics, which is validated by extensive empirical analysis. LSTR achieves state-of-the-art performance on three standard online action detection benchmarks, THUMOS'14, TVSeries, and HACS Segment. Code has been made available at: <https://xumingze0308.github.io/projects/lstr>.

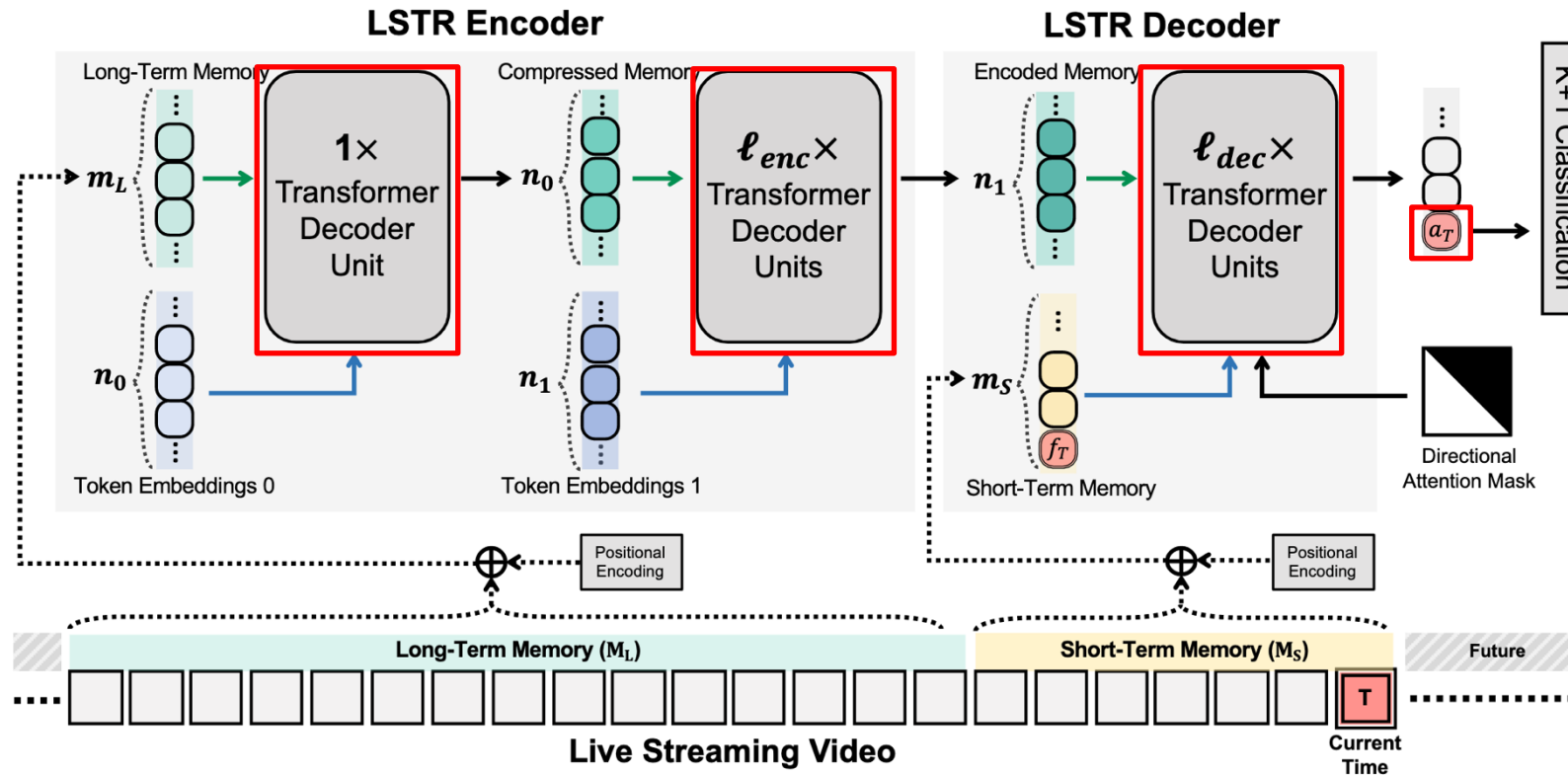
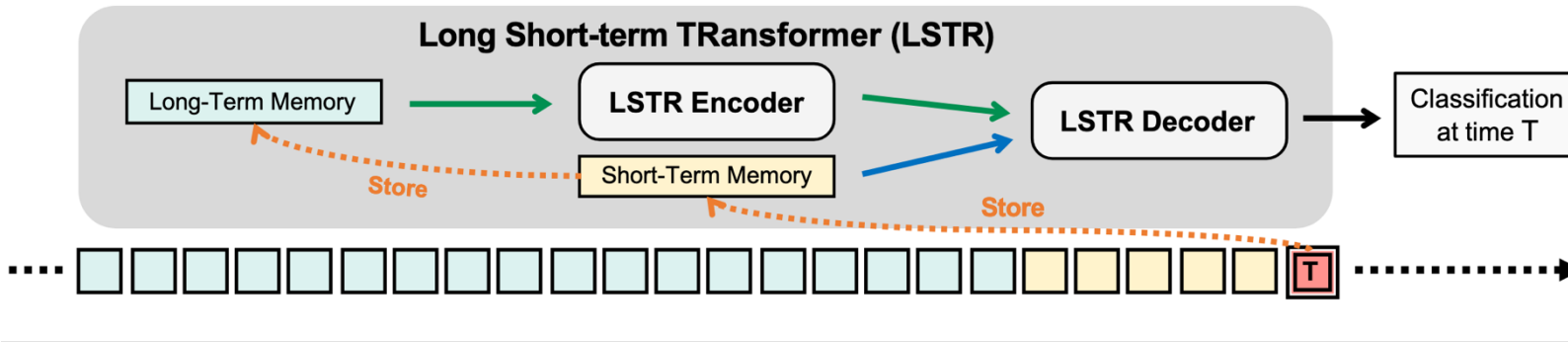
Long Short-Term Transformer (LSTR)



LSTR tries to address this problem by using **two memories**:

- **Long-term**: all past observations, compressed with an encoder to reduce size
- **Short-term**: within a small sliding window, used to «index» content from the long-term memory

Long Short-Term Transformer (LSTR)

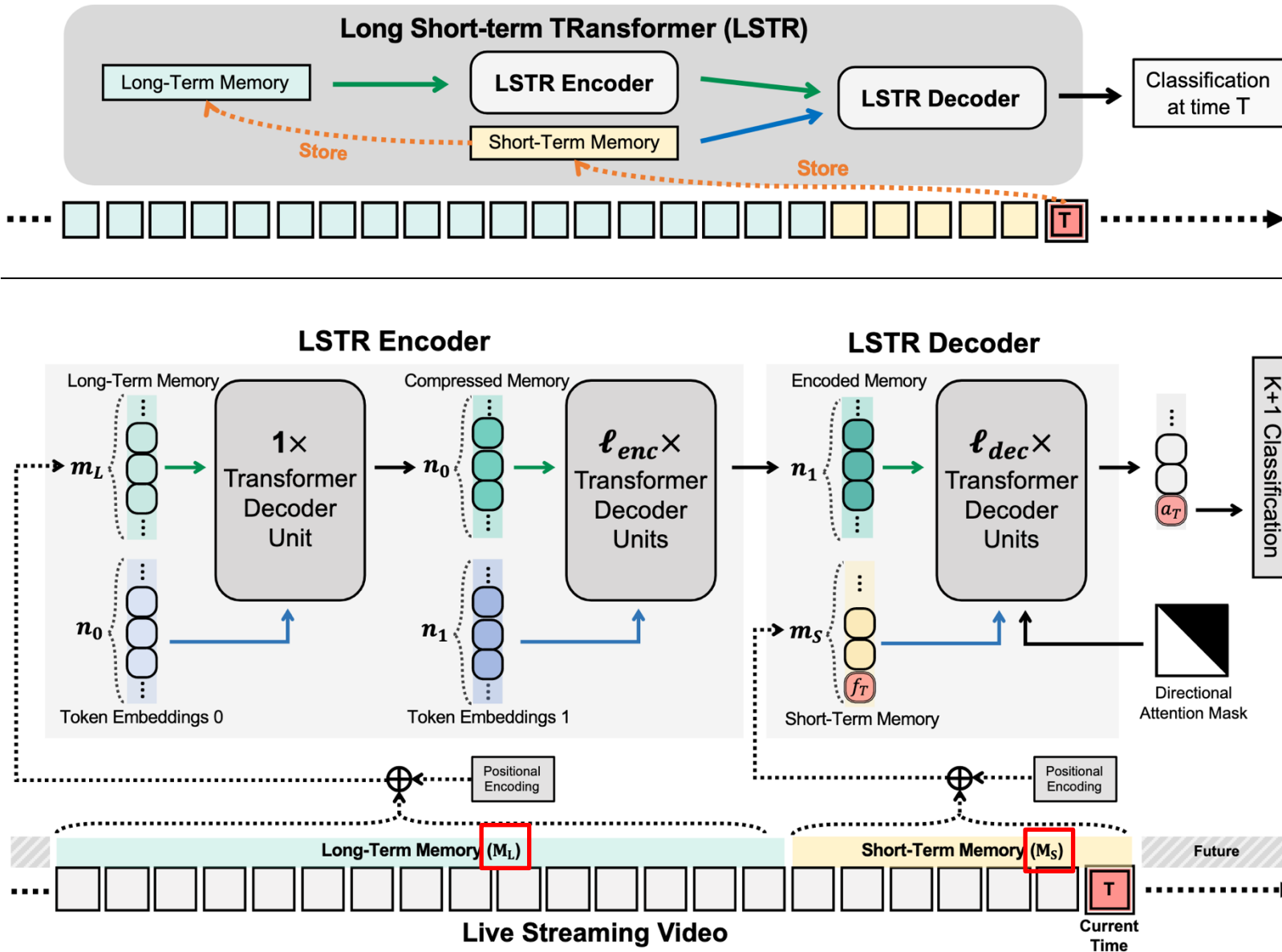


- **Compression** is achieved using a **single decoder layer** and a set of learned token embeddings
- The compressed memory is further transformed with a **multi-layer decoder**
- The short-term memory is then used to **retrieve information from the encoded memory** using another multi-layer decoder
- The prediction corresponding to the current time is taken as **current prediction**

Long Short-Term Transformer (LSTR)

- Long and short memory **sizes are fixed to L and S** respectively
- Compression is optimized by **caching previous keys and values** to save computation
- This approach allows the model to work in a «**streaming**» fashion, where elements from the **short-term memory progressively transit into the long-term memory**, which is compressed and kept as a reference.

Still discards old values



Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention

Angelos Katharopoulos^{1 2} Apoorv Vyas^{1 2} Nikolaos Pappas³ François Fleuret^{2 4 *}

Abstract

Transformers achieve remarkable performance in several tasks but due to their quadratic complexity, with respect to the input’s length, they are prohibitively slow for very long sequences. To address this limitation, we express the self-attention as a linear dot-product of kernel feature maps and make use of the associativity property of matrix products to reduce the complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$, where N is the sequence length. We show that this formulation permits an iterative implementation that dramatically accelerates autoregressive transformers and reveals their relationship to recurrent neural networks. Our *linear transformers* achieve similar performance to vanilla transformers and they are up to 4000x faster on autoregressive prediction of very long sequences.

by the global receptive field of self-attention, which processes contexts of N inputs with a quadratic memory and time complexity $\mathcal{O}(N^2)$. As a result, in practice transformers are slow to train and their context is *limited*. This disrupts temporal coherence and hinders the capturing of long-term dependencies. Dai et al. (2019) addressed the latter by attending to memories from previous contexts albeit at the expense of computational efficiency.

Lately, researchers shifted their attention to approaches that increase the context length without sacrificing efficiency. Towards this end, Child et al. (2019) introduced sparse factorizations of the attention matrix to reduce the self-attention complexity to $\mathcal{O}(N\sqrt{N})$. Kitaev et al. (2020) further reduced the complexity to $\mathcal{O}(N \log N)$ using locality-sensitive hashing. This made scaling to long sequences possible. Even though the aforementioned models can be efficiently trained on large sequences, they do not speed-up autoregressive inference.

Similarity-Based Attention

Standard Attention

$$Q = xW_Q,$$

$$K = xW_K,$$

$$V = xW_V,$$

$$A_l(x) = V' = \text{softmax} \left(\frac{QK^T}{\sqrt{D}} \right) V.$$

Equivalent To

$$V'_i = \frac{\sum_{j=1}^N \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^N \text{sim}(Q_i, K_j)}.$$

$$\text{sim}(q, k) = \exp \left(\frac{q^T k}{\sqrt{D}} \right)$$

Linearized Attention and Kernel View

Equivalent To

$$V'_i = \frac{\sum_{j=1}^N \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^N \text{sim}(Q_i, K_j)}.$$

$$\text{sim}(q, k) = \exp\left(\frac{q^T k}{\sqrt{D}}\right)$$

In practice, we obtain a valid attention choosing sim as any valid kernel function $k(x, y) = \mathbb{R}^{2 \times F} \rightarrow R_+$ (F feature size).

Given such a kernel with feature representation ϕ (i.e., such that $k(x, y) = \phi(x)\phi(y)$), we can re-write attention as follows:

$$V'_i = \frac{\sum_{j=1}^N \phi(Q_i)^T \phi(K_j) V_j}{\sum_{j=1}^N \phi(Q_i)^T \phi(K_j)},$$

Transformers are RNNs

Linear Complexity and Elu Function

Given such a kernel with a feature representation $\phi(x)$ we can rewrite equation 2 as follows,

$$V'_i = \frac{\sum_{j=1}^N \phi(Q_i)^T \phi(K_j) V_j}{\sum_{j=1}^N \phi(Q_i)^T \phi(K_j)}, \quad (4)$$

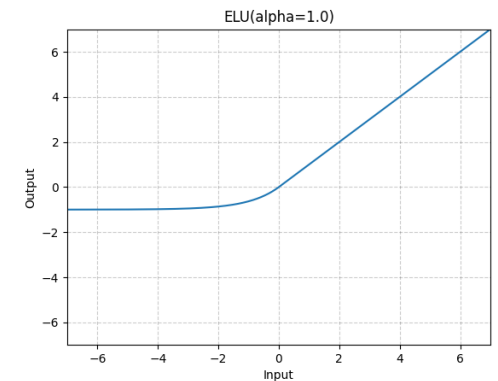
and then further simplify it by making use of the associative property of matrix multiplication to

$$V'_i = \frac{\phi(Q_i)^T \sum_{j=1}^N \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^N \phi(K_j)}. \quad (5)$$

This is much more efficient ($O(N)$ vs $O(N^2)$) as we can compute $\sum_j \phi(K_j) V_j^T$ and $\sum_j \phi(K_j)$ once and re-use it for every query.

Authors use:

$$\phi(x) = \text{elu}(x) + 1,$$



Causal Masking

Causal masking for online processing changes attention form this form:

$$V'_i = \frac{\sum_{j=1}^N \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^N \text{sim}(Q_i, K_j)}.$$

To this form:

$$V'_i = \frac{\sum_{j=1}^i \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^i \text{sim}(Q_i, K_j)}.$$

Following the kernel view:

$$V'_i = \frac{\phi(Q_i)^T \sum_{j=1}^i \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^i \phi(K_j)}.$$

Or Alternatively:

$$V'_i = \frac{\phi(Q_i)^T S_i}{\phi(Q_i)^T Z_i} \quad \text{with} \quad \begin{aligned} S_i &= \sum_{j=1}^i \phi(K_j) V_j^T, \\ Z_i &= \sum_{j=1}^i \phi(K_j), \end{aligned}$$

Transformers are RNNs

Linear transformers with causal masking

$$V'_i = \frac{\phi(Q_i)^T S_i}{\phi(Q_i)^T Z_i}$$

with

$$S_i = \sum_{j=1}^i \phi(K_j) V_j^T,$$

$$Z_i = \sum_{j=1}^i \phi(K_j),$$

Recurrent View

$$s_0 = 0,$$

$$z_0 = 0,$$

$$s_i = s_{i-1} + \phi(x_i W_K) (x_i W_V)^T,$$

$$z_i = z_{i-1} + \phi(x_i W_K),$$

$$y_i = f_l \left(\frac{\phi(x_i W_Q)^T s_i}{\phi(x_i W_Q)^T z_i} + x_i \right).$$

State Space Models and Mamba (or the return of RNNs)

State Space Models

$$\dot{\mathbf{x}}(t) = \mathbf{A}(t)\mathbf{x}(t) + \mathbf{B}(t)\mathbf{u}(t)$$

$$\mathbf{y}(t) = \mathbf{C}(t)\mathbf{x}(t) + \mathbf{D}(t)\mathbf{u}(t)$$

Where:

$\mathbf{x}(\cdot)$ is called the "state vector", $\mathbf{x}(t) \in \mathbb{R}^n$;

$\mathbf{y}(\cdot)$ is called the "output vector", $\mathbf{y}(t) \in \mathbb{R}^q$;

$\mathbf{u}(\cdot)$ is called the "input (or control) vector", $\mathbf{u}(t) \in \mathbb{R}^p$;

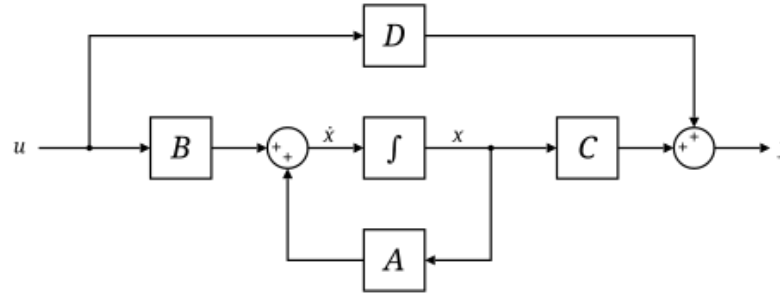
$\mathbf{A}(\cdot)$ is the "state (or system) matrix", $\dim[\mathbf{A}(\cdot)] = n \times n$,

$\mathbf{B}(\cdot)$ is the "input matrix", $\dim[\mathbf{B}(\cdot)] = n \times p$,

$\mathbf{C}(\cdot)$ is the "output matrix", $\dim[\mathbf{C}(\cdot)] = q \times n$,

$\mathbf{D}(\cdot)$ is the "feedthrough (or feedforward) matrix" (in cases where the system model does not have a direct feedthrough, $\mathbf{D}(\cdot)$ is the zero matrix), $\dim[\mathbf{D}(\cdot)] = q \times p$,

$$\dot{\mathbf{x}}(t) := \frac{d}{dt}\mathbf{x}(t).$$



- In **control engineering**, a state space model describes how a system evolves over time by defining (two equations):
 - **State evolution**: how the internal state changes
 - **Observation model**: how the output (what we observe) relates to the internal state
- The internal state of the state space model is very similar to the concept of «**hidden state**» in an RNN

A, B, C, and D are usually set by hand depending on our knowledge of the system.

$$\dot{\mathbf{x}}(t) = \mathbf{A}(t)\mathbf{x}(t) + \mathbf{B}(t)\mathbf{u}(t)$$

$$\mathbf{y}(t) = \mathbf{C}(t)\mathbf{x}(t)$$

State Space Models

Derivative of $\mathbf{x}$ with
respect to time
«how the state
evolves in time»

Ideally, if we know $\dot{\mathbf{x}}(t)$, we can
compute the state at future times

$$\mathbf{x}(t') = \mathbf{x}(t) + \int_t^{t'} \dot{\mathbf{x}}(\tau) d\tau$$

$$\begin{aligned} \overbrace{\dot{\mathbf{x}}(t)}^{\text{Derivative of } \mathbf{x} \text{ with respect to time}} &= \mathbf{A}(t) \overbrace{\mathbf{x}(t)}^{\text{Current state}} + \mathbf{B}(t) \overbrace{\mathbf{u}(t)}^{\text{Current input}} \\ \mathbf{y}(t) &= \mathbf{C}(t) \underbrace{\mathbf{x}(t)}_{\text{Current state}} + \mathbf{D}(t) \underbrace{\mathbf{u}(t)}_{\text{Current input}} \end{aligned}$$

- $\mathbf{A}(t)$, $\mathbf{B}(t)$, $\mathbf{C}(t)$, and $\mathbf{D}(t)$ are generally **time-dependent**
- However, we can simplify and **drop the dependence on time**, considering these as constant matrices
- We hence obtain a **Linear Time Invariant (LTI)** system where matrices do not evolve with time

$$\begin{aligned} \dot{\mathbf{x}}(t) &= \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) \\ \mathbf{y}(t) &= \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t) \end{aligned}$$

Continuous State Space model Example



force applied with the gas pedal to make the car move

$u(t)$



$x_1(t)$ position

$x_2(t)$ velocity

m mass of the car

$$F_f(t) = -b \cdot x_2(t)$$

Friction force (proportional to velocity)

To build a state space model, we now need to find the derivatives of the state. For $\dot{x}_1(t)$ it's easy, as velocity is the derivative of position:

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} \begin{matrix} \leftarrow \text{position} \\ \leftarrow \text{velocity} \end{matrix}$$

System state

$$\dot{x}_1(t) = x_2(t)$$

how position evolves with time

For $\dot{x}_2(t)$, recall $F = m \cdot a$ (mass times acceleration), so:

$$\underbrace{m \cdot \dot{x}_2(t)}_{\text{acceleration (derivative of velocity)}} = \underbrace{u(t)}_{\text{gas}} - \underbrace{b \cdot x_2(t)}_{\text{friction}} \Rightarrow \underbrace{\dot{x}_2(t) = \frac{u(t)}{m} - \frac{b}{m} x_2(t)}_{\text{how velocity evolves with time and input } u(t)}$$

We finally have this system describing the dynamics of the car:

$$\begin{cases} \dot{x}_1(t) = x_2(t) \\ \dot{x}_2(t) = \frac{u(t)}{m} - \frac{b}{m} x_2(t) \end{cases}$$

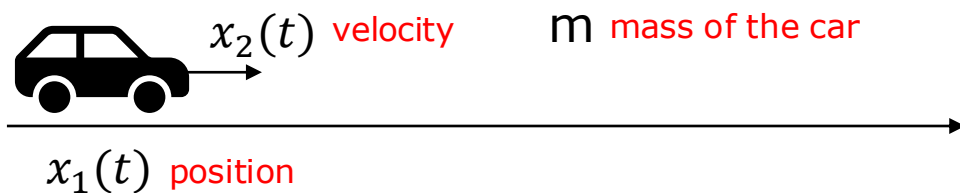
- Imagine a car moving in a straight line.
- We want to model its position and velocity over time.
- The system state $x(t)$ will be made of:
 - $x_1(t)$: position at each moment in time
 - $x_2(t)$: velocity at each moment in time
- We also assume that the car has friction (air or road) proportional to the car's velocity with constant $b > 0$
- The car's state is controlled by the input acceleration $u(t)$
 - Note that the position and velocity at each time step are not just determined by the acceleration, but also by the state of the system (past velocity and position) which act as a sort of «memory» of the system.

Continuous State Space model Example



force applied with the gas pedal to make the car move

$u(t)$



$$F_f(t) = -b \cdot x_2(t)$$

Friction force (proportional to velocity)

To build a state space model, we now need to find the derivatives of the state. For $\dot{x}_1(t)$ it's easy, as velocity is the derivative of position:

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} \begin{matrix} \leftarrow \text{position} \\ \leftarrow \text{velocity} \end{matrix}$$

System state

$$\dot{x}_1(t) = x_2(t)$$

how position evolves with time

For $\dot{x}_2(t)$, recall $F = m \cdot a$ (mass times acceleration), so:

$$\underbrace{m \cdot \dot{x}_2(t)}_{\text{acceleration (derivative of velocity)}} = \underbrace{u(t)}_{\text{gas}} - \underbrace{b \cdot x_2(t)}_{\text{friction}} \Rightarrow \underbrace{\dot{x}_2(t) = \frac{u(t)}{m} - \frac{b}{m} x_2(t)}_{\text{how velocity evolves with time and input } u(t)}$$

We finally have this system describing the dynamics of the car:

$$\begin{cases} \dot{x}_1(t) = x_2(t) \\ \dot{x}_2(t) = \frac{u(t)}{m} - \frac{b}{m} x_2(t) \end{cases}$$

We can rewrite the system as follows:

$$\dot{x}(t) = \underbrace{\begin{bmatrix} 0 & 1 \\ 0 & -\frac{b}{m} \end{bmatrix}}_A x(t) + \underbrace{\begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}}_B u(t) \quad \text{System of equations in matrix form}$$

Also, since we are interested in knowing only the position as output, our output equation will be:

$$y(t) = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_C x(t) \quad \text{Selects position and discards velocity}$$

In practice, we defined the following time-invariant state space model:

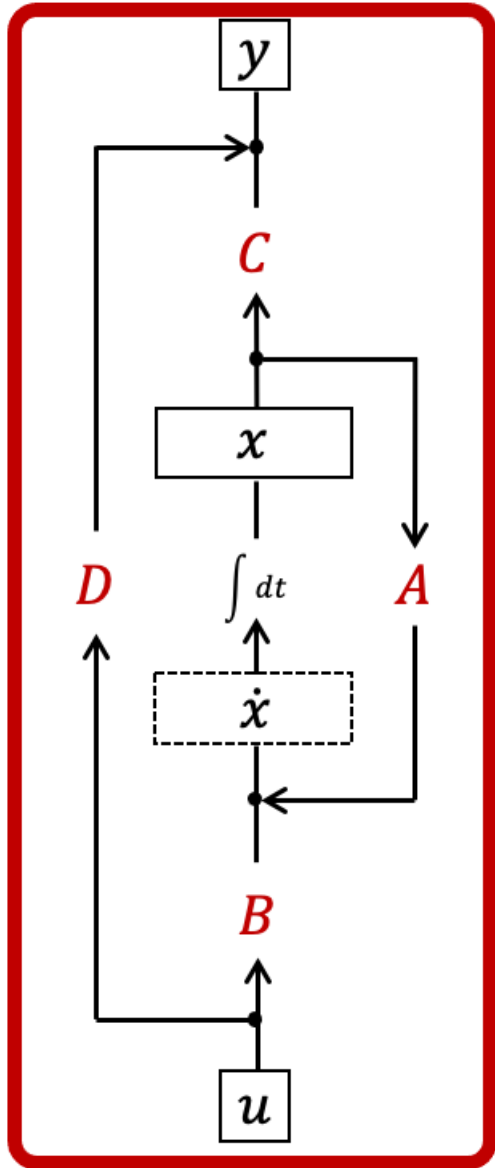
$$\dot{x}(t) = Ax(t) + Bu(t)$$

$$y(t) = Cx(t) + Du(t)$$

Where:

$$A = \begin{bmatrix} 0 & 1 \\ 0 & -\frac{b}{m} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad D = 0$$

Continuous State Space Models



$$\dot{\mathbf{x}}(t) = \mathbf{A}(t)\mathbf{x}(t) + \mathbf{B}(t)\mathbf{u}(t)$$

$$\mathbf{y}(t) = \mathbf{C}(t)\mathbf{x}(t) + \mathbf{D}(t)\mathbf{u}(t)$$

- We can see continuous state space models as shown on the left.
- The input u is multiplied by the matrix B
- The current state x is multiplied by A
- These are added to compute the derivative $\dot{x}$
- Integration allows us to compute a new value of x
- The state x is multiplied by C to obtain the output
- Finally, u is multiplied by D and the result is added to the output
- The last term can be seen as a skip connection from input to output (similar to ResNet)

Discretized State Space Model



force applied with the gas pedal to make the car move

$u(t)$



$x_1(t)$ position

$x_2(t)$ velocity

m mass of the car

$$\dot{x}(t) = \underbrace{\begin{bmatrix} 0 & 1 \\ 0 & -\frac{b}{m} \end{bmatrix}}_A x(t) + \underbrace{\begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}}_B u(t) \quad y(t) = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_C x(t)$$

The state space model defines the dynamics of the car with respect to the input force in a continuous space. However, we are treating t as a continuous quantity, while for practical applications (e.g., simulations), we need to reason in discrete terms. For instance, we may want to compute the position of the car every Δt seconds.

To do so, we need to discretize our state space model. This can be done in different ways. One easy way is to use the Euler method.

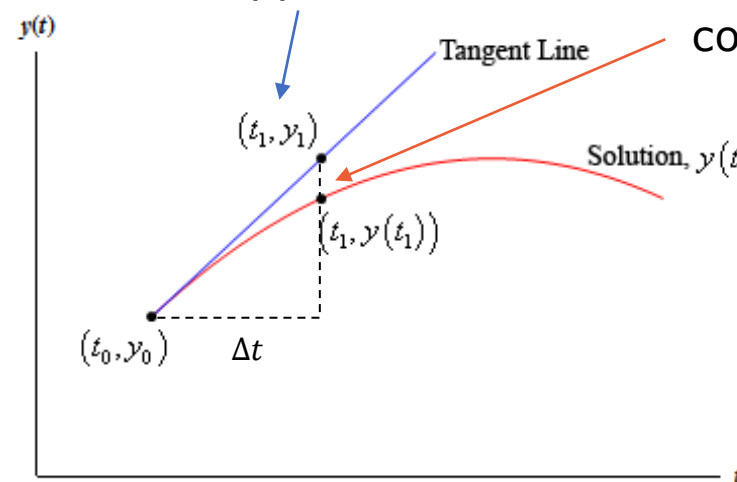
In general, let's assume:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = 0$$

We want to compute

$$y(t_1 = t_0 + \Delta t)$$

Euler approximation



correct, analytical solution

$$\begin{aligned} y(t_1 = t_0 + \Delta t) &= y(t_0) + f(t_0, y_0)(t_1 - t_0) \\ &= y(t_0) + f(t_0, y_0)\Delta t \end{aligned}$$

If Δt is small enough, the approximation is not too bad

Discretized State Space Model



force applied with the gas pedal to make the car move

$u(t)$



$x_1(t)$ position

$x_2(t)$ velocity

m mass of the car

$$\dot{x}(t) = \underbrace{\begin{bmatrix} 0 & 1 \\ 0 & -\frac{b}{m} \end{bmatrix}}_A x(t) + \underbrace{\begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}}_B u(t) \quad y(t) = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_C x(t)$$

We now assume discretized steps k happening at a distance of Δt . Applying Euler's rule to our system, we obtain:

$$\begin{aligned} \dot{x}_1(t) &= x_2(t) \\ \dot{x}_2(t) &= \frac{u(t)}{m} - \frac{b}{m} x_2(t) \end{aligned}$$



$$\begin{aligned} x_1[k+1] &= x_1[k] + \Delta t \cdot x_2[k] \\ x_2[k+1] &= x_2[k] + \Delta t \cdot \left(-\frac{b}{m} x_2[k] + \frac{1}{m} u[k] \right) \end{aligned}$$

Equivalently:

$$x[k+1] = A_d x[k] + B_d u[k]$$

$$A_d = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 - \frac{b}{m} \Delta t \end{bmatrix}, \quad B_d = \begin{bmatrix} 0 \\ \frac{\Delta t}{m} \end{bmatrix}$$

$$y[k] = C x[k], \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

In general terms, we convert a continuous state space model discretizing the matrices with some formula:

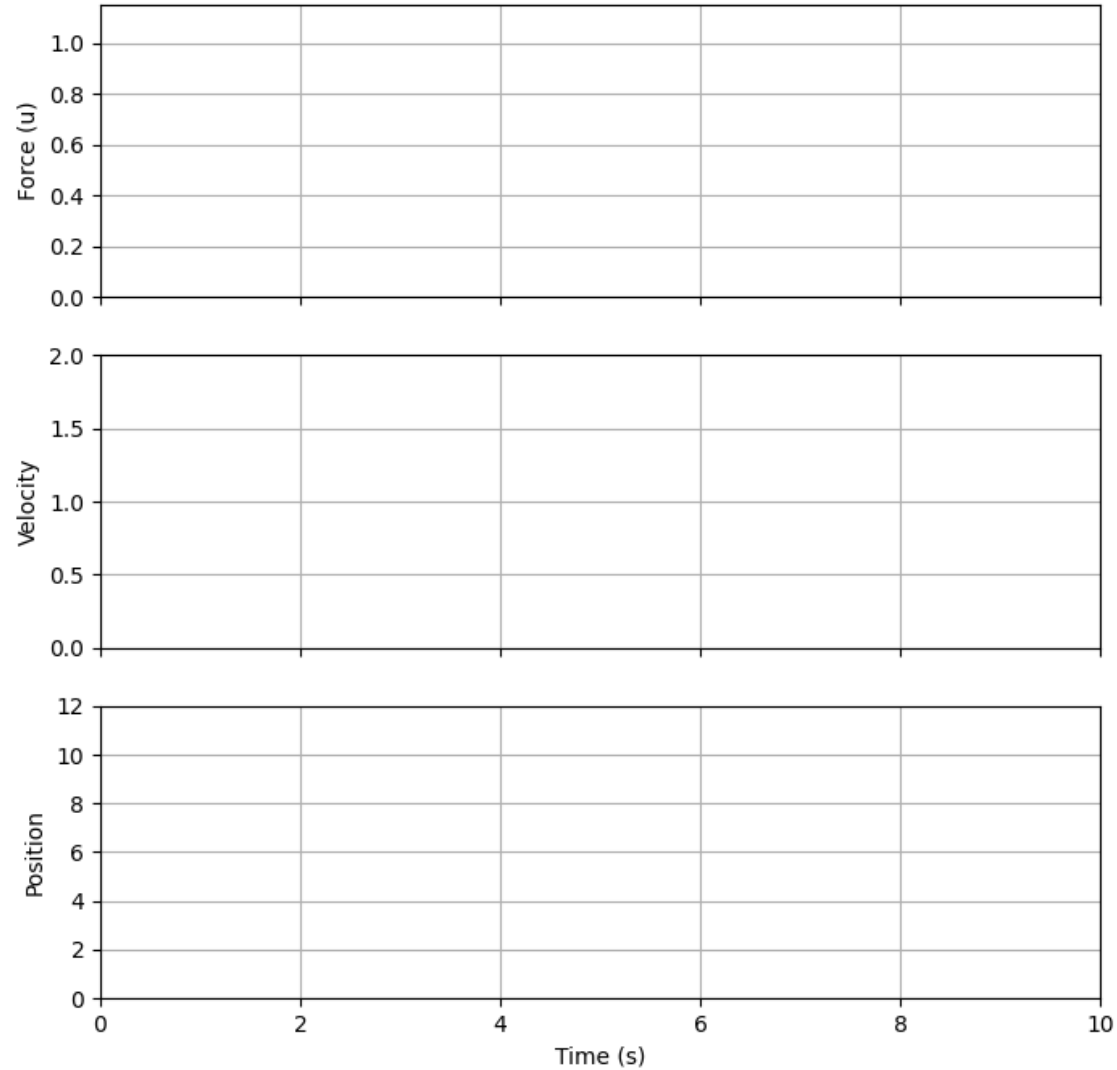
$$A_d = \text{discretize}_A(A, \Delta t), \quad B_d = \text{discretize}_B(B, A, \Delta t)$$

If we use the forward Euler rule (but we could use other rules):

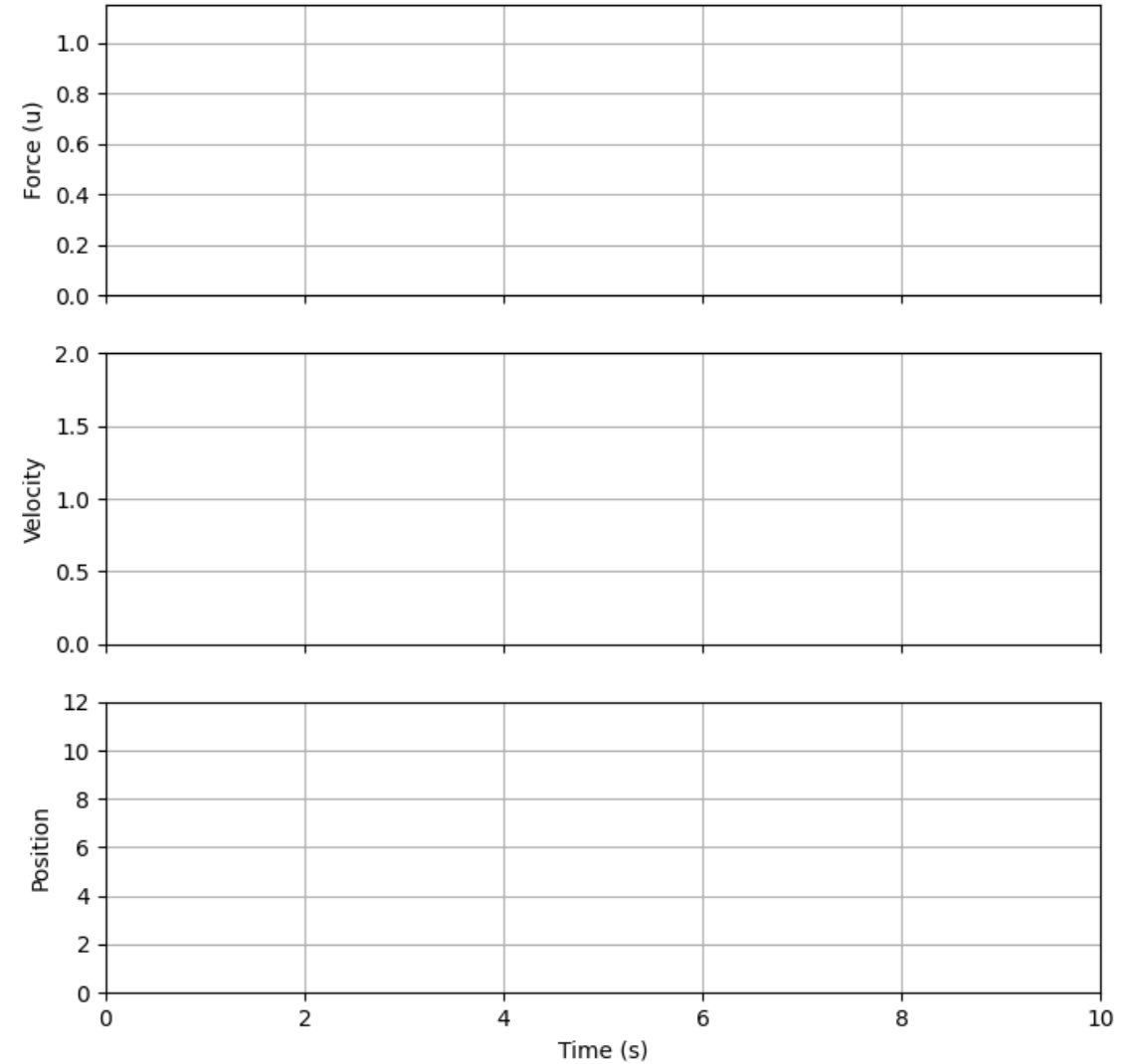
$$A_d \approx I + \Delta t \cdot A, \quad B_d \approx \Delta t \cdot B$$

Discretized State Space Model - Simulation

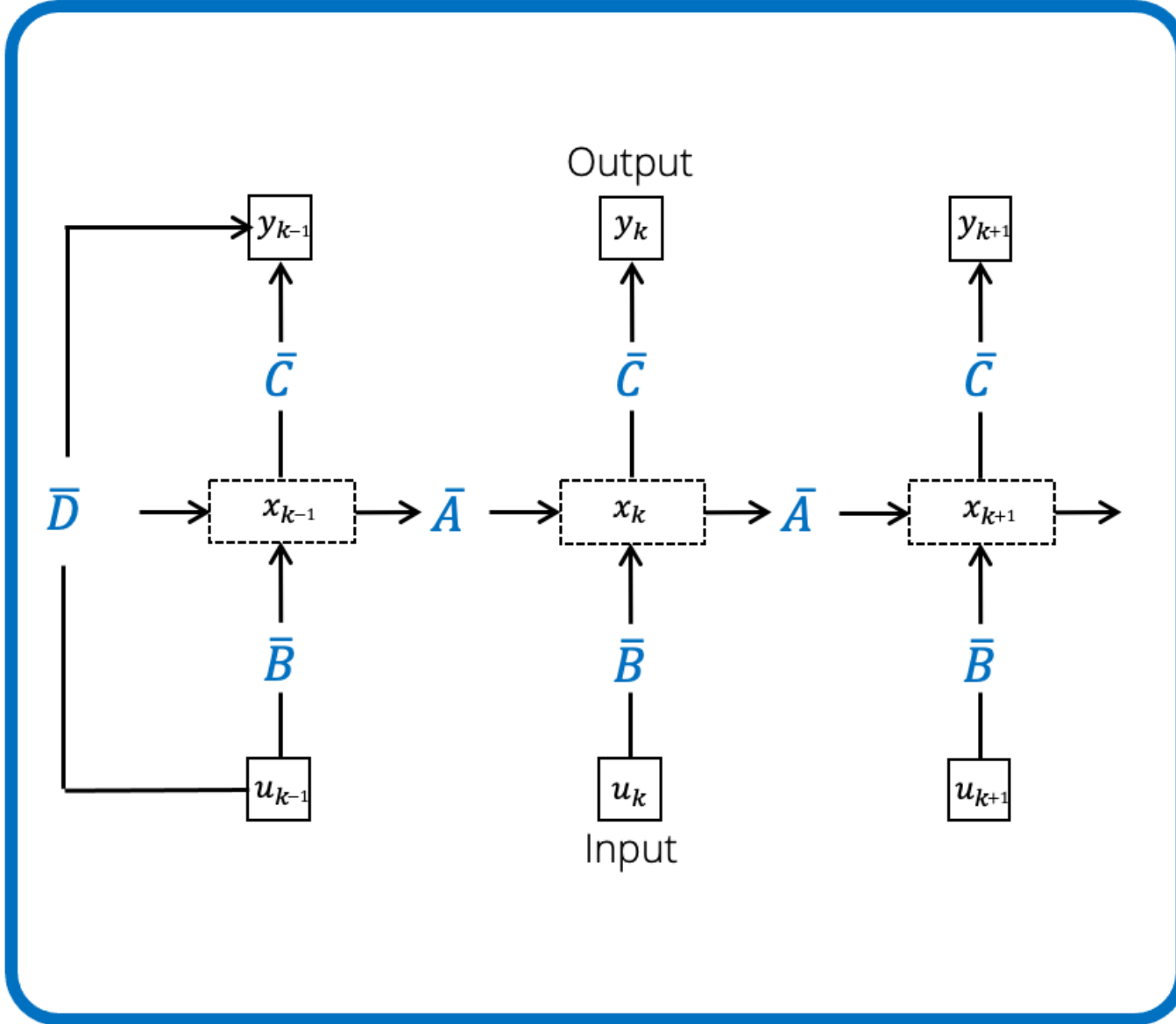
$\Delta t = 0.1$



$\Delta t = 0.5$



Discretized State Space Models



$$x_{k+1} = A_d x_k + B_d u_k \quad (\text{state update})$$

$$y_k = C_d x_k + D_d u_k \quad (\text{output})$$

- Discretized state space models can be seen as shown on the left
- This is similar to the continuous view with A and B used to compute the next state, C used to compute the output, and D allowing to implement a skip connection

<https://hazyresearch.stanford.edu/blog/2022-01-14-s4-3>

State Space Models

$$x_{k+1} = A_d x_k + B_d u_k \quad (\text{state update})$$

$$y_k = C_d x_k + \boxed{D_d u_k} \quad (\text{output})$$

This is usually omitted

VS

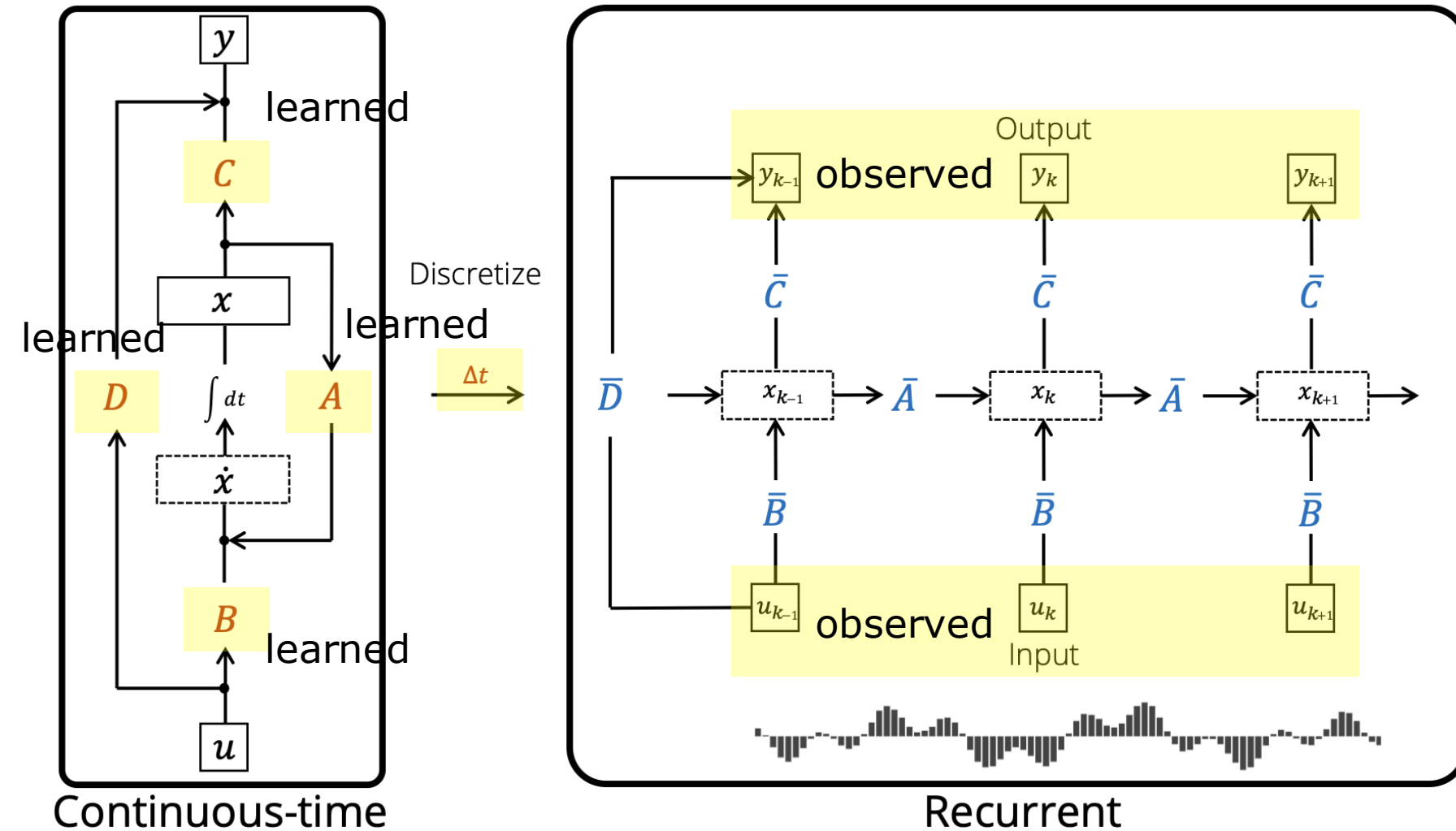
Recurrent Neural Networks

$$h_{k+1} = \boxed{\sigma}(W_h h_k + W_x u_k + \boxed{b_h}) \quad (\text{hidden state})$$

$$y_k = W_y h_k + \boxed{b_y} \quad (\text{output})$$

Very similar, apart from bias terms, **non-linearity**, and a skip-connection

Neural State Space Models



- Similar to RNNs, **rather than setting the A , B , C , and D matrices by hand, we can learn them!**
- Since the **discrete matrices can be computed from the continuous ones** (e.g., A vs $\bar{A}$) with known differentiable formulas, we can actually learn the continuous versions
- Note that different SSMs can use **different discretizations**

State Space Model Computation

Algorithm 1 SSM (S4)

Input: $x : (B, L, D)$

Output: $y : (B, L, D)$

1: $A : (D, N) \leftarrow \text{Parameter}$

- ▷ Represents structured $N \times N$ matrix

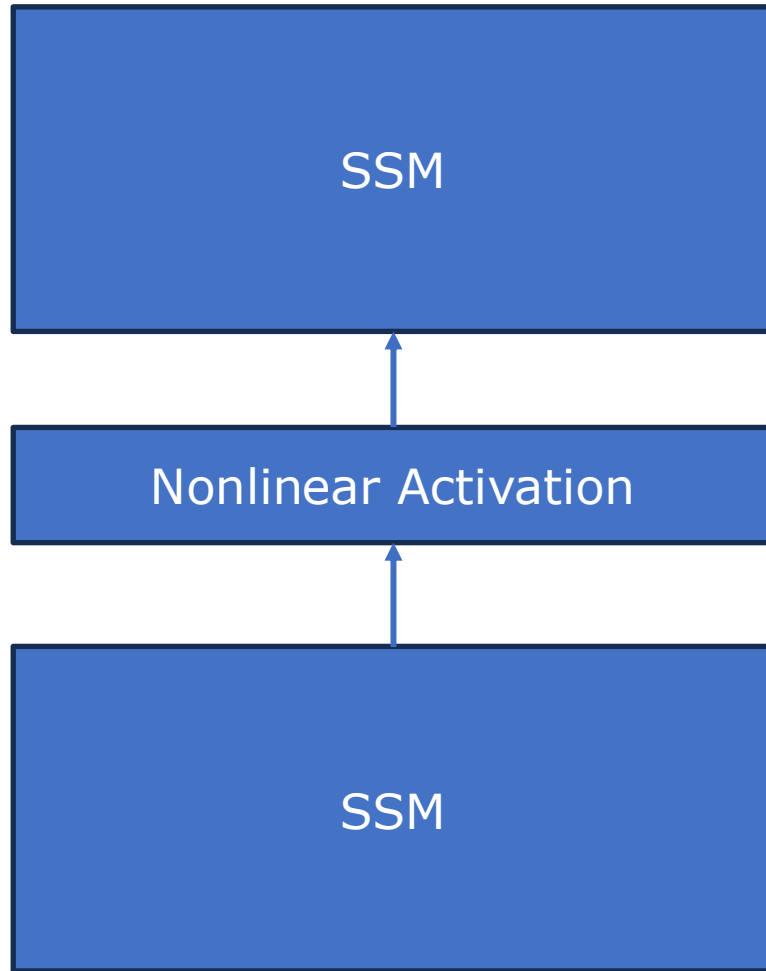
2: $B : (D, N) \leftarrow \text{Parameter}$ 3: $C : (D, N) \leftarrow \text{Parameter}$ 4: $\Delta : (D) \leftarrow \tau_{\Delta}(\text{Parameter})$ 5: $\overline{A}, \overline{B} : (D, N) \leftarrow \text{discretize}(\Delta, A, B)$ 6: $y \leftarrow \text{SSM}(\overline{A}, \overline{B}, C)(x)$

- ▷ Time-invariant: recurrence or convolution

```
7: return  $y$ 
```

- An SSM learns continuous A , B , and C matrices, as well as a discretization parameter Δ ;
- A and B are discretized using known formulas and used to compute the SSM;
- This can be computed recurrently (typically at test time), but it can also be computed with a convolution to speed up training.

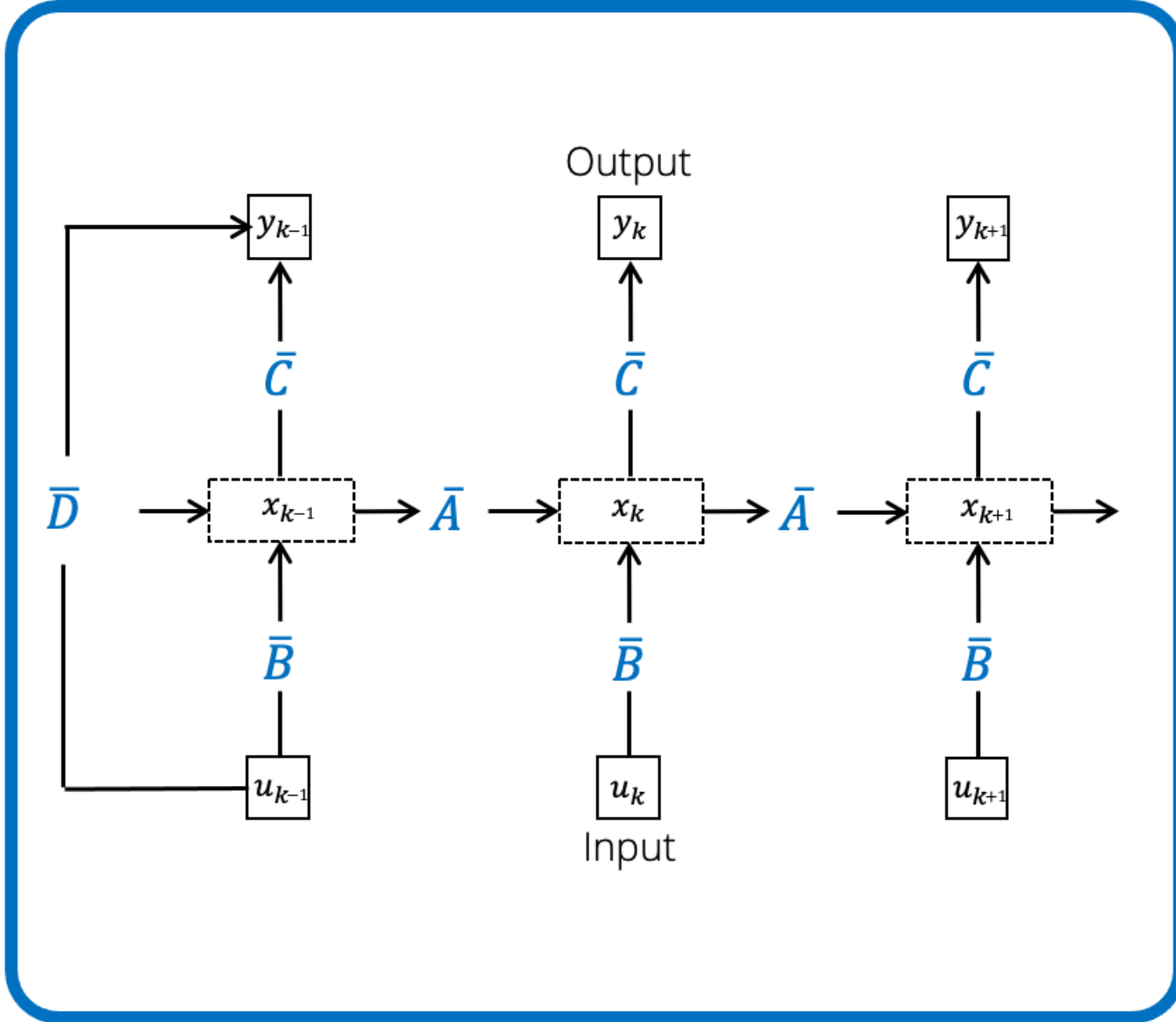
So... are SSMs just linear models?



$$x_{k+1} = A_d x_k + B_d u_k \quad (\text{state update})$$
$$y_k = C_d x_k + D_d u_k \quad (\text{output})$$

- While SSMs are linear, we can easily make them more expressive by stacking SSM layers and putting non-linearities in between
- This is the same approach followed by Transformers
- Indeed, attention is a linear operation at its core, with final MLPs adding non-linearities
- By stacking multiple layers in a deep architectures, SSMs can model nonlinear relationships

Recurrence, isn't this slow?



- We know from RNNs, that they are hard to parallelize and hence slow at training time;
- Indeed, in order to compute the output of the model at time k , we need the output obtained at time $k-1$ due to the recurrence;
- Don't we have the same problem with SSMs?

<https://hazyresearch.stanford.edu/blog/2022-01-14-s4-3>

Convolutional View of SSMs

It actually turns out that SSMs can be computed using a convolution:

$$x_k = \bar{\mathbf{A}}x_{k-1} + \bar{\mathbf{B}}u_k$$

$$y_k = \bar{\mathbf{C}}x_k$$

In general

$$y_k = \bar{\mathbf{C}}\bar{\mathbf{A}}^k\bar{\mathbf{B}}u_0 + \bar{\mathbf{C}}\bar{\mathbf{A}}^{k-1}\bar{\mathbf{B}}u_1 + \dots + \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}u_{k-1} + \bar{\mathbf{C}}\bar{\mathbf{B}}u_k$$

$$y = \bar{\mathbf{K}} * u \quad \bar{\mathbf{K}} \in \mathbb{R}^L = (\bar{\mathbf{C}}\bar{\mathbf{B}}, \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}, \dots, \bar{\mathbf{C}}\bar{\mathbf{A}}^{L-1}\bar{\mathbf{B}})$$

Efficient if we pre-compute the kernel!

Step 0: $x_0 = \bar{\mathbf{B}}u_0$

Step 0: $y_0 = \bar{\mathbf{C}}x_0 = \bar{\mathbf{C}}\bar{\mathbf{B}}u_0$

Step 1: $x_1 = \bar{\mathbf{A}}x_0 + \bar{\mathbf{B}}u_1 = \bar{\mathbf{A}}\bar{\mathbf{B}}u_0 + \bar{\mathbf{B}}u_1$

Step 1: $y_1 = \bar{\mathbf{C}}x_1 = \bar{\mathbf{C}}(\bar{\mathbf{A}}\bar{\mathbf{B}}u_0 + \bar{\mathbf{B}}u_1) = \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}u_0 + \bar{\mathbf{C}}\bar{\mathbf{B}}u_1$

Step 2: $x_2 = \bar{\mathbf{A}}x_1 + \bar{\mathbf{B}}u_2 = \bar{\mathbf{A}}(\bar{\mathbf{A}}\bar{\mathbf{B}}u_0 + \bar{\mathbf{B}}u_1) + \bar{\mathbf{B}}u_2 = \bar{\mathbf{A}}^2\bar{\mathbf{B}}u_0 + \bar{\mathbf{A}}\bar{\mathbf{B}}u_1 + \bar{\mathbf{B}}u_2$

Step 2: $y_2 = \bar{\mathbf{C}}x_2 = \bar{\mathbf{C}}(\bar{\mathbf{A}}^2\bar{\mathbf{B}}u_0 + \bar{\mathbf{A}}\bar{\mathbf{B}}u_1 + \bar{\mathbf{B}}u_2) = \bar{\mathbf{C}}\bar{\mathbf{A}}^2\bar{\mathbf{B}}u_0 + \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}u_1 + \bar{\mathbf{C}}\bar{\mathbf{B}}u_2$

Copying, Selective Copying, and Induction Heads

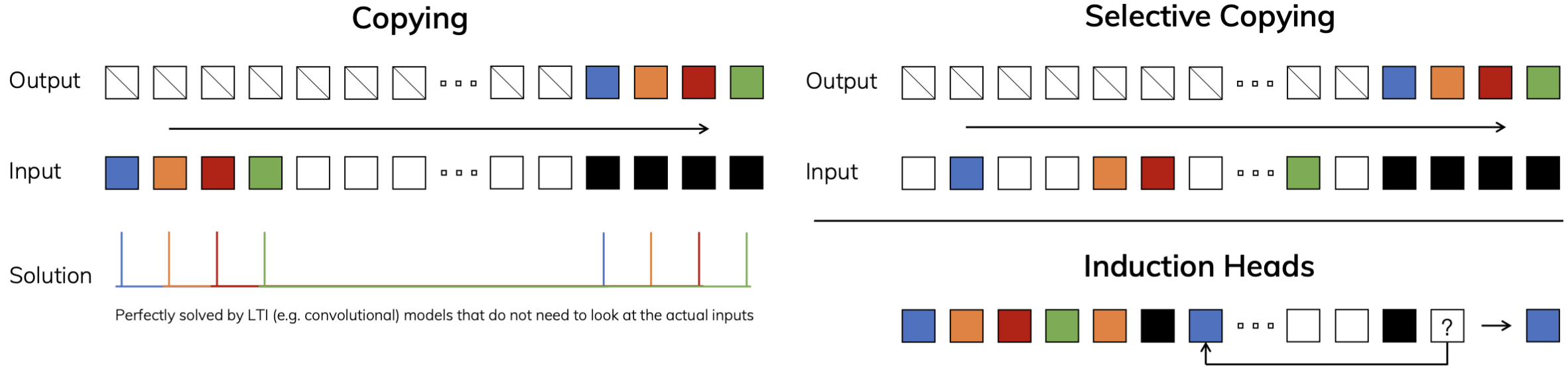
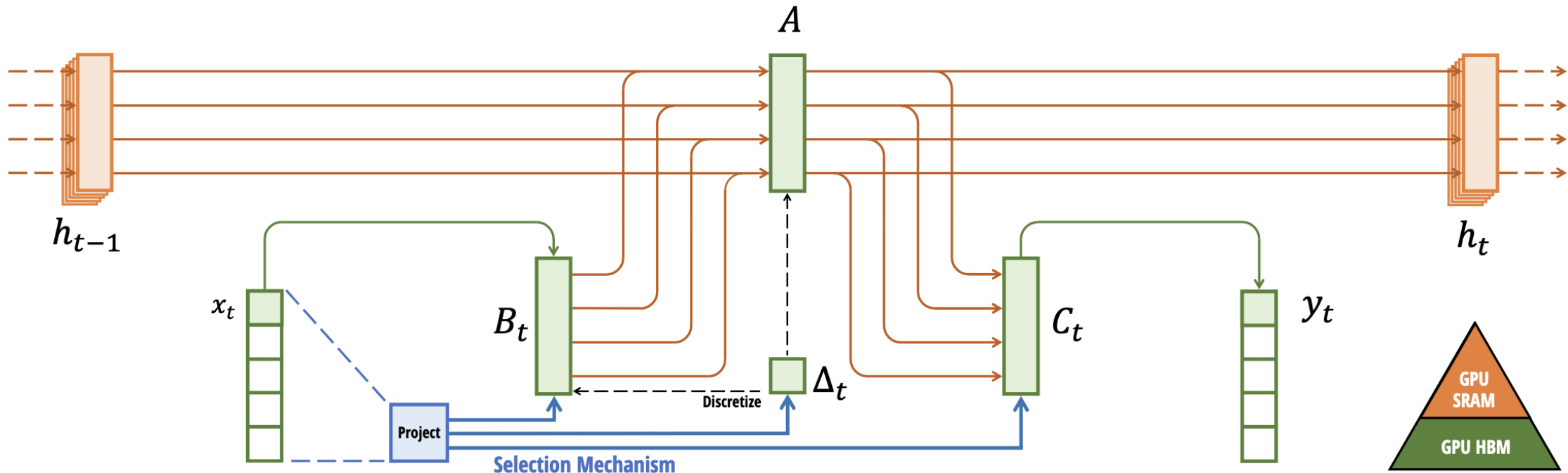


Figure 2: (Left) The standard version of the Copying task involves constant spacing between input and output elements and is easily solved by time-invariant models such as linear recurrences and global convolutions. (Right Top) The Selective Copying task has random spacing in between inputs and requires time-varying models that can *selectively* remember or ignore inputs depending on their content. (Right Bottom) The Induction Heads task is an example of associative recall that requires retrieving an answer based on context, a key ability for LLMs.

MAMBA: Selective SSM computed with a scan (S6)

Main idea: make B and C time-varying and depending on the inputs. These matrices are multiplied by the input, so making them time-varying allows to use them as gates filtering out irrelevant information (e.g., white spaces in selective copying).



The selection mechanism

Algorithm 1 SSM (S4)

Input: $x : (B, L, D)$

Output: $y : (B, L, D)$

1: $A : (D, N) \leftarrow \text{Parameter}$

▷ Represents structured $N \times N$ matrix

2: $B : (D, N) \leftarrow \text{Parameter}$

3: $C : (D, N) \leftarrow \text{Parameter}$

4: $\Delta : (D) \leftarrow \tau_{\Delta}(\text{Parameter})$

5: $\bar{A}, \bar{B} : (D, N) \leftarrow \text{discretize}(\Delta, A, B)$

6: $y \leftarrow \text{SSM}(\bar{A}, \bar{B}, C)(x)$

▷ Time-invariant: recurrence or convolution

7: **return** y

Algorithm 2 SSM + Selection (S6)

Input: $x : (B, L, D)$

Output: $y : (B, L, D)$

1: $A : (D, N) \leftarrow \text{Parameter}$

▷ Represents structured $N \times N$ matrix

2: $B : (B, L, N) \leftarrow s_B(x)$

3: $C : (B, L, N) \leftarrow s_C(x)$

4: $\Delta : (B, L, D) \leftarrow \tau_{\Delta}(\text{Parameter} + s_{\Delta}(x))$

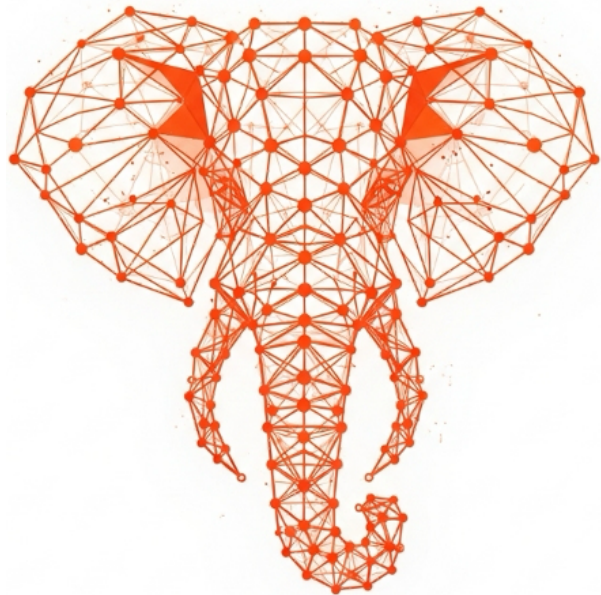
5: $\bar{A}, \bar{B} : (B, L, D, N) \leftarrow \text{discretize}(\Delta, A, B)$

6: $y \leftarrow \text{SSM}(\bar{A}, \bar{B}, C)(x)$

▷ **Time-varying:** recurrence (*scan*) only

7: **return** y

- Differently from S4, here B , C and Δt are time-varying (they gained an L dimension);
- We obtain time varying discretized $\bar{A}_t$ (A is one, but discretized many times), $\bar{B}_t$, $\bar{C}_t$
- In practice, MAMBA uses a selective scan operation to make this efficient



Deep Learning

Advanced Models & Methods

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Laurea Magistrale in Informatica

Dip. di Matematica e Informatica

Università di Catania

The End